Curso PostgreSQL 18 arquitectura monitoreo

create un curso de postgresql 18 de arquitectura, monitoreo y optimizacion
Leer 9 páginas web
PostgreSQL 18 ha llegado con cambios fundamentales en su arquitectura de I/O y capacidades de monitoreo. He estructurado este curso para que puedas pasar de entender las nuevas bases teóricas a aplicar técnicas concretas de optimización y usar las métricas avanzadas que ahora están disponibles.

Curso: PostgreSQL 18 - Arquitectura, Monitoreo y Optimización

Audiencia Objetivo

- **Administradores de Bases de Datos (DBAs)** que planean migrar o actualizar a PostgreSQL 18.
- **DevOps/SREs** encargados de desplegar y mantener bases de datos en producción (especialmente en Kubernetes).
- **Desarrolladores Backend** que desean exprimir el rendimiento de la nueva versión.

Modalidad

- **Teórico-Práctico**: 60% teoría (arquitectura interna), 40% práctica (consultas SQL, comandos `psql`, configuración).
- **Nivel**: Avanzado.

Módulo 1: Arquitectura Interna y Nuevos Fundamentos de I/O

Objetivo: Comprender los cambios estructurales de PostgreSQL 18, centrándose en el nuevo subsistema de I/O Asíncrono (AIO) y su impacto en la gestión de memoria y procesos.

1.1. Evolución de la Arquitectura de I/O

- **Contexto**: En versiones anteriores, PostgreSQL usaba I/O síncrono (`read`/`write`) dependiendo del *readahead* del sistema operativo.
- **Limitación**: El SO no entiende los patrones de acceso interno de la BD (ej. índices vs secuenciales) .

1.2. Async I/O (AIO) en Profundidad

- **Concepto**: Capacidad de lanzar múltiples solicitudes de I/O en paralelo sin esperar a que cada una termine secuencialmente.
- **Mecanismo**: Uso de `io_uring` en Linux para DMA (Direct Memory Access), liberando la CPU mientras se espera el disco .
- **Configuración Clave**: Nuevo parámetro `io_method` (`worker`, `io_uring`, `sync`).
- **Casos de Uso**:
- *Secuenciales (2x-3x mejora)*: `pg_dump`, `ANALYZE`, `VACUUM`.
- *Limitaciones actuales*: Todavía no implementado para *Index Scans* en esta versión .

1.3. Planificador y Estadísticas

- **Retención de Estadísticas**: `pg_upgrade` ahora preserva las estadísticas del planificador, eliminando el "periodo de frio" post-actualización .
- **Skip Scan**: Optimización para índices B-Tree multi-columna cuando se omite la columna prefijo .

1.4. Caso Práctico: Verificando AIO

- Configurar `postgresql.conf`:

  conf

  ```
  io_method = 'io_uring'          # Linux 5.1+
  shared_buffers = 25%            # Ajustar según RAM total
  effective_io_concurrency = 200  # Valores altos para AIO
  ```
- Ejecutar un `EXPLAIN (ANALYZE, BUFFERS)` en un `SELECT` masivo vs un `UPDATE` para ver la diferencia en la gestión de buffers.

Módulo 2: Monitoreo Avanzado y Observabilidad

Objetivo: Utilizar las nuevas vistas y herramientas de estadísticas para diagnosticar cuellos de botella en I/O, WAL y concurrencia.

2.1. Estadísticas por Proceso (Per-Backend)

- **Novedad**: Posibilidad de aislar el rendimiento de una conexión específica.
- **Funciones Nuevas**:
- `pg_stat_get_backend_io(pid)`: I/O del proceso (bytes vs páginas).
- `pg_stat_get_backend_wal(pid)`: Cantidad de WAL generado por sesión.
- `pg_backend_memory_contexts`: Nuevas columnas `path` y `type` (Slab, Generation) para depurar fugas de memoria .

2.2. Estadísticas Acumuladas (pg_stat_*)

- **pg_stat_io**: Ahora reporta en **bytes** (no páginas), más intuitivo. Incluye stats de WAL.
- **pg_stat_all_tables**: Nuevas columnas de tiempo total empleado en VACUUM y ANALYZE (autovacuum manual).
- **pg_stat_subscription_stats**: Nuevos contadores para conflictos lógicos (ej. `confl_insert_exists`, `confl_update_missing`) .

2.3. EXPLAIN de Próxima Generación

- **Auto-Buffers**: `EXPLAIN (ANALYZE)` muestra `Buffers:` por defecto.
- **Estadísticas de I/O**: `EXPLAIN (ANALYZE, VERBOSE)` ahora muestra tiempos de lectura promedio y WAL generado.

2.4. Laboratorio: Diagnóstico

- Identificar consultas que hacen más I/O usando `pg_stat_statements` (actualizado para agrupar mejor consultas con `IN`).
- Usar `pg_stat_get_backend_io()` para encontrar procesos bloqueados en I/O.

Módulo 3: Optimización del Motor y Ajuste de Índices

Objetivo: Aplicar técnicas de optimización específicas de PG 18, incluyendo nuevos tipos de datos y estrategias de indexación.

3.1. Optimización de Tipos de Datos

- **UUIDv7 vs UUIDv4**:
- *Problema v4*: Aleatorio -> Fragmentación de índices.
- *Solución v7*: Basado en timestamp (monotónico) -> Mejor localidad y compresión .
- *Práctica*:

  sql

  ```
  CREATE TABLE users (id UUID PRIMARY KEY DEFAULT uuidv7(), name TEXT);
  ```
- **Columnas Virtuales**: Uso de `GENERATED VIRTUAL` (por defecto) para ahorrar almacenamiento vs `STORED` .

3.2. Nuevas Estrategias de Indexación

- **Skip Scan**:
- *Escenario*: Índice `(status, created_at)`.
- *Query*: `SELECT * FROM orders WHERE created_at > '2025-01-01'`.

- *Explicación*: PG 18 puede saltarse `status` si hay pocos valores distintos .
- **Paralelismo en GIN**: `CREATE INDEX CONCURRENTLY ... WITH (parallel_workers = 4)`; mejora la creación de índices para JSONB y Full-Text Search .

3.3. Optimización de DDL y Migraciones

- **NOT NULL con NOT VALID**:

1. `ALTER TABLE t ALTER COLUMN c SET NOT NULL NOT VALID;` (instantáneo).
2. Rellenar datos null.
3. `ALTER TABLE t VALIDATE CONSTRAINT ...;` (sin lock pesado) .

- **Foreign Keys en Particionadas**: Ahora soportan `NOT VALID` y validación por partición .

3.4. Taller: Refactorización

- Migrar una tabla con PK UUIDv4 a UUIDv7.
- Revisar el tamaño del índice antes/después usando `pg_stat_all_indexes`.

Módulo 4: Seguridad, Autenticación y Mantenimiento

Objetivo: Configurar entornos seguros usando OAuth 2.0 y gestionar el nuevo comportamiento de mantenimiento.

4.1. Autenticación Moderna

- **OAuth 2.0 / SSO**: Nueva capacidad nativa (`pg_hba.conf` método `oauth`). Permite tokens JWT y flujos interactivos .
- **Deprecación de MD5**: Postgres 18 deshabilita MD5 por defecto; forzar **SCRAM-SHA-256**.
- **FIPS Mode**: Nuevo parámetro `ssl_tls13_ciphers` para cumplimiento gubernamental .

4.2. VACUUM y Transacciones

- **Freeze Proactivo**: El `VACUUM` ahora congela páginas de forma más agresiva para prevenir *transaction wraparound*.
- **Monitoreo Wraparound**: Uso de `pg_datasentinel` o las nuevas métricas en `pg_stat_progress_vacuum` para estimar ETAs de riesgo .

4.3. Upgrades sin Downtime

- **pg_upgrade --swap**: Cambia los directorios en lugar de copiar archivos, reduciendo drásticamente el tiempo de inactividad .

4.4. Configuración de Producción

- Activar `track_cost_delay_timing` con precaución (alto overhead en sistemas lentos) .
- Data Checksums activados por defecto en `initdb` .

Módulo 5: SQL Avanzado y Temporal DB

Objetivo: Explotar las nuevas extensiones SQL para modelado de datos complejos y series temporales.

5.1. Nuevas Cláusulas SQL

- **RETURNING con OLD/NEW**:

  sql

  ```
  UPDATE employees SET salary = salary * 1.1 WHERE id = 10
  RETURNING OLD.salary AS before, NEW.salary AS after;
  ``` [citation:1]
- **MERGE** (Upsert mejorado): Ahora más integrado con el nuevo planificador.

5.2. Restricciones Temporales

- **Periodos sin Overlap**:

  sql

  ```
  CREATE TABLE reservation (
      room INT,
      during daterange,
      PRIMARY KEY (room, during WITHOUT OVERLAPS)
  );
  ```
  Garantiza que no se reserve la misma sala al mismo tiempo .

5.3. Full-Text Search y JSONB

- Uso de `GIN` indexes con parallel build.
- Nueva collation `PG_UNICODE_FAST` para comparaciones de texto más rápidas .

5.4. Proyecto Final

- Diseñar un sistema de *Auditoría Temporal* usando `WITHOUT OVERLAPS` y `RETURNING OLD/NEW` para trackear cambios de estado.

Recursos y Herramientas Complementarias

- **pg_datasentinel v1.0**: Extensión para monitoreo en contenedores (cgroups, memoria por backend) .
- **pg_stat_monitor 2.3.1**: Compatible con PG 18 para métricas de WAL y workers paralelos .

- **Repositorio de Referencia**: "Formation PostgreSQL 18" (21 capítulos) para profundizar en teoría .

Resumen de Novedades Técnicas (Cheat Sheet)

Área	Feature Clave	Impacto
Arquitectura	Async I/O (`io_uring`)	2x-3x en sequential scans
Monitoreo	Stats por backend (I/O y WAL)	Diagnóstico de fugas
Optimización	Skip Scan & UUIDv7	Mejor performance de índices
Operaciones	`pg_upgrade --swap`	Downtime casi nulo
SQL	`RETURNING OLD/NEW`	Simplifica audit logs
Seguridad	OAuth 2.0 nativo	Integración SSO directa

desarrollame el modulo 1 y con todo esto que te paso, para postgresql 18:Módulo 1: Arquitectura de PostgreSQL (6 horas) 1.1. Introducción a la arquitectura Modelo cliente/servidor Procesos postmaster, background writer, WAL writer, autovacuum, stats collector Memoria compartida: shared buffers, wal buffers, work_mem, maintenance_work_mem 1.2. Estructura de almacenamiento Tablespaces físicos y lógicos Estructura de una base de datos: OID, pg_class, pg_attribute Segmentos, extensiones, bloques (8 KB por defecto) TOAST (The Oversized-Attribute Storage Technique) 1.3. Índices internos B-Tree, Hash, GIN, GiST, BRIN, SP-GiST Cuando usar cada tipo (ejemplos prácticos) 1.4. MVCC (Multiversion Concurrency Control) Cómo PostgreSQL maneja concurrencia sin bloqueos de lectura xmin, xmax, cid, ctid Visibilidad y limpieza de tuplas muertas (dead tuples) Ejercicio: Identificar procesos del sistema, ver configuración de memoria y crear un tablespace

Excelente. A continuación desarrollaré el **Módulo 1 completo** para PostgreSQL 18, integrando las novedades arquitectónicas que mencioné anteriormente (especialmente AIO y las nuevas capacidades de monitoreo por proceso) con los conceptos clásicos que solicitaste.

Módulo 1: Arquitectura de PostgreSQL 18 (6 horas)

Objetivos del Módulo

- Comprender el modelo de procesos y memoria de PostgreSQL 18.
- Identificar los nuevos componentes introducidos en esta versión (Async I/O, estadísticas por backend).
- Analizar la estructura de almacenamiento físico y lógico.
- Entender MVCC y su impacto en rendimiento.

- Aplicar conceptos mediante ejercicios prácticos.

1.1. Introducción a la Arquitectura (1.5 horas)

Modelo Cliente/Servidor en PG 18

PostgreSQL utiliza una arquitectura **proceso por conexión** (fork-based):

- Cada cliente se conecta a un **proceso backend dedicado**.
- No utiliza hilos ligeros (threads), lo que proporciona aislamiento pero mayor consumo de recursos.

Flujo de conexión típico:

text

```
Cliente → Puerto 5432 → Postmaster → Fork nuevo backend → Atiende consultas
```

Procesos Fundamentales de PG 18

Proceso	Función en PG 18	Novedad
Postmaster	Proceso padre que escucha conexiones y lanza backends	Sin cambios
Background Writer	Escribe buffers sucios a disco de forma periódica	Ahora puede usar AIO para writes asíncronos
WAL Writer	Escribe registros WAL desde el buffer WAL a disco	Mejoras en compresión y agrupamiento
Autovacuum	Limpia tuplas muertas y actualiza estadísticas	Integración con `pg_stat_progress_vacuum` mejorada
Stats Collector	Recolecta estadísticas de actividad	**NUEVO**: Recolecta por backend individual
Checkpointer	Realiza puntos de control (checkpoints)	Coordinación mejorada con AIO
Logger	Maneja logs del sistema	Ahora puede rotar logs sin reiniciar

Memoria Compartida en PG 18

Shared Buffers (Configuración crítica)

- **Propósito**: Caché de páginas de datos (tablas e índices).
- **Recomendación PG 18**: 25% de RAM total (versiones anteriores sugerían 15-25%).
- **Monitoreo nuevo**:

  sql

```
-- Ver uso de buffers por tipo de operación (nueva columna en pg_stat_io)
SELECT backend_type, reads, writes, read_bytes, write_bytes
FROM pg_stat_io
WHERE io_object = 'relation';
```

WAL Buffers

- **Tamaño**: 16 MB por defecto (aumentado desde 8 MB en PG 15).
- **Función**: Buffer circular para registros WAL antes de escribir a disco.
- **Novedad**: PG 18 permite `wal_buffers = auto` (ajusta hasta 64 MB).

Work_mem (Por operación)

- **Uso**: Ordenamientos, hash joins, bitmap scans.
- **Peligro**: Si es muy alto → consumo excesivo de RAM.
- **Monitoreo**: `pg_stat_get_backend_memory(pid)` para ver uso real.

Maintenance_work_mem

- **Uso**: VACUUM, CREATE INDEX, ADD FOREIGN KEY.
- **PG 18 mejora**: Ahora el autovacuum puede usar hasta el 10% de esto para trabajos en paralelo.

Ejemplo Práctico 1.1: Ver procesos y memoria

sql

```
-- Ver todos los procesos de PostgreSQL (desde el sistema operativo)
SELECT pid, usename, application_name, state, query
FROM pg_stat_activity;

-- Ver configuración actual de memoria (en bytes)
SHOW shared_buffers;
SHOW work_mem;
SHOW maintenance_work_mem;

-- Ver estadísticas de I/O por backend (nuevo en PG 18)
SELECT
    b.pid,
    b.usename,
    io.read_bytes,
    io.write_bytes,
    io.reads,
    io.writes
FROM pg_stat_activity b
JOIN pg_stat_get_backend_io(b.pid) io ON true
WHERE b.state = 'active';
```

1.2. Estructura de Almacenamiento (1.5 horas)

Tablespaces: Físicos vs Lógicos

Concepto:

- **Tablespace lógico**: Nombre simbólico definido en PostgreSQL.
- **Tablespace físico**: Directorio en el sistema de archivos.

Estrategias comunes en PG 18:

- Separar índices en discos SSD rápidos.
- Ubicar tablas grandes (logs, auditoría) en discos lentos y baratos.
- Usar `pg_upgrade --swap` para migrar tablespaces sin downtime.

Creación y uso:

sql

```
-- Crear tablespace en un directorio específico
CREATE TABLESPACE fast_ssd LOCATION '/mnt/ssd/postgres/data';

-- Crear tabla usando ese tablespace
CREATE TABLE logs_rapidas (id serial, data text) TABLESPACE fast_ssd;

-- Mover una tabla existente
ALTER TABLE usuarios SET TABLESPACE fast_ssd;
```

Estructura Interna: OID, pg_class, pg_attribute

OID (Object Identifier):

- Identificador único de 32 bits para cada objeto (tabla, índice, función).
- PG 18 mantiene OIDs solo para system catalogs por defecto (ahorro de espacio).

pg_class: Catálogo maestro de todas las tablas e índices.

sql

```
-- Ver información de una tabla
SELECT relname, relowner, reltuples, relpages, reltoastrelid
FROM pg_class
WHERE relname = 'nombre_tabla';
```

pg_attribute: Define las columnas de cada tabla.

sql

```
-- Ver columnas de una tabla con su tipo
SELECT attname, atttypid::regtype, atttypmod, attnotnull
FROM pg_attribute
WHERE attrelid = 'usuarios'::regclass
  AND attnum > 0;  -- excluir system columns
```

Segmentos, Extensiones y Bloques

Jerarquía física:

text

```
Base de datos → Tabla → Segmento → Extensión → Bloque (8 KB)
```

Componente	Tamaño	Descripción
Bloque (Block)	8 KB	Unidad mínima de I/O (configurable en

Componente	Tamaño	Descripción
		compilación)
Extensión (Extent)	128 KB (16 bloques)	Crecimiento incremental de tablas
Segmento (Segment)	1 GB (máx 32 GB)	Archivo físico (relfilenode)

PG 18 novedad: Mejor manejo de extensiones con AIO para crecimiento rápido.

Ver archivos físicos:

sql

```
-- Obtener el relfilenode actual de una tabla
SELECT relname, relfilenode, reltablespace
FROM pg_class
WHERE relname = 'mi_tabla';

-- En el sistema operativo:
-- ls -la $PGDATA/base/16384/relfilenode_num
```

TOAST (The Oversized-Attribute Storage Technique)

Propósito: Almacenar valores grandes (>2 KB) fuera de la fila principal.

Estrategias TOAST en PG 18:

- **PLAIN**: No comprimir ni externalizar (solo para tipos fijos).
- **EXTENDED**: Comprimir primero, externalizar si sigue siendo grande.
- **EXTERNAL**: Externalizar sin comprimir (mejor para acceso parcial).
- **MAIN**: Comprimir pero no externalizar (raro).

Configuración por columna:

sql

```
ALTER TABLE productos
ALTER COLUMN descripcion SET STORAGE EXTENDED;
```

Monitoreo TOAST (nuevo en PG 18):

sql

```
-- Ver tablas TOAST y su tamaño
SELECT
    t.relname AS tabla_principal,
    toast.relname AS tabla_toast,
    pg_table_size(toast.oid) AS toast_tamaño
FROM pg_class t
JOIN pg_class toast ON t.reltoastrelid = toast.oid
WHERE t.relkind = 'r';
```

1.3. Índices Internos (1.5 horas)

Tipos de Índices en PG 18

Tipo	Mejor para	Tamaño típico	Novedad PG 18
B-Tree	Igualdad, rango, ordenamiento	2-4x datos	Skip scan nativo
Hash	Igualdad exacta	Similar a B-Tree	Soporte para valores NULL
GIN	Arrays, JSONB, texto completo	Grande (10x+)	Construcción paralela
GiST	Datos geométricos, rangos	Mediano	Mejoras en kNN search
BRIN	Tablas muy grandes, correlación temporal	Muy pequeño (1% de tabla)	Soporte para más operadores
SP-GiST	Datos con agrupamiento natural	Mediano	Sin cambios mayores

Guía de selección (casos prácticos)

Caso 1: Búsqueda por rango de fechas en logs

sql

```
-- Opción 1: BRIN (si los datos están ordenados por fecha)
CREATE INDEX logs_fecha_brin ON logs USING brin (fecha_registro);
-- Tamaño: ~128 KB por 1 GB de datos

-- Opción 2: B-Tree tradicional
CREATE INDEX logs_fecha_btree ON logs (fecha_registro);
-- Tamaño: ~200 MB por 1 GB de datos
```

Caso 2: Búsqueda en JSONB con muchos campos

sql

```
-- GIN index para acceso rápido a campos internos
CREATE INDEX productos_datos_gin ON productos USING gin (datos_jsonb);
```

Caso 3: Búsqueda geoespacial

sql

```
CREATE INDEX ubicaciones_gist ON ubicaciones USING gist (geom);
```

Skip Scan (Nuevo en PG 18)

Problema resuelto: Antes, un índice `(status, created_at)` no ayudaba a `WHERE created_at > '2025-01-01'` sin filtrar `status`.

Solución: PG 18 puede "saltarse" la primera columna si tiene pocos valores únicos.

sql

```
-- Índice multicolumna
CREATE INDEX pedidos_estado_fecha ON pedidos (estado, fecha_pedido);
```

```
-- Query que ahora usa Skip Scan
EXPLAIN SELECT * FROM pedidos
WHERE fecha_pedido > '2026-01-01';
-- Plan: "Skip Scan using pedidos_estado_fecha"
```

Ver estadísticas de índices

sql

```sql
-- Tamaño de índices
SELECT schemaname, tablename, indexname,
       pg_size_pretty(pg_indexes_size(schemaname||'.'||tablename)) AS
tamaño_total
FROM pg_indexes
WHERE tablename = 'mi_tabla';

-- Uso de índices (PG 18 mejorado)
SELECT schemaname, tablename, indexname,
       idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
WHERE idx_scan = 0;  -- Índices no utilizados
```

1.4. MVCC (Multiversion Concurrency Control) (1.5 horas)

Cómo PostgreSQL maneja concurrencia sin bloqueos de lectura

Principio fundamental: Lectores nunca bloquean escritores, y viceversa.

Mecanismo:

- Cada modificación crea una **nueva versión** de la tupla (fila).
- Las versiones viejas se marcan como "muertas" pero permanecen físicamente.
- Las transacciones ven la versión activa en el momento de su inicio (`snapshot`).

Campos de control de tupla (HeapTupleHeader)

Campo	Tamaño	Descripción
xmin	4 bytes	Transacción que creó esta versión
xmax	4 bytes	Transacción que eliminó/marcó esta versión
cmin/cmax	4 bytes	Command ID dentro de la transacción
ctid	6 bytes	Ubicación física (bloque, offset) de la versión actual

Ejemplo visual:

text

```text
Tabla original:                 Después de UPDATE:
[id=1, name='Ana', xmin=100]    [id=1, name='Ana', xmin=100, xmax=102] (muerta)
                                [id=1, name='Maria', xmin=102, ctid=(0,2)]
```

Visibilidad y tuplas muertas

Proceso de visibilidad:

1. Si `xmin` está abortada → invisible
2. Si `xmin` está en curso → visible solo para esa transacción
3. Si `xmax` está commit y es anterior al snapshot → invisible
4. Si `xmax` está abortada → visible (fue eliminada sin commit)

Dead Tuples (tuplas muertas):

- Versiones que ya no son visibles para ninguna transacción.
- Causan **bloat** (hinchazón de la tabla).
- Limpiadas por **VACUUM** o **Autovacuum**.

Monitoreo MVCC en PG 18

sql

```
-- Ver dead tuples por tabla (nuevas columnas en pg_stat_all_tables)
SELECT
    schemaname,
    tablename,
    n_live_tup AS tuplas_vivas,
    n_dead_tup AS tuplas_muertas,
    last_vacuum,
    last_autovacuum,
    -- Nuevo en PG 18:
    vacuum_count AS veces_vacunada,
    autovacuum_count AS veces_autovacunada,
    vacuum_time AS tiempo_total_vacuum,
    autovacuum_time AS tiempo_total_autovacuum
FROM pg_stat_all_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC;
```

Configuración crítica de Autovacuum

sql

```
-- Ver configuración actual
SHOW autovacuum_vacuum_threshold;       -- Default: 50 dead tuples
SHOW autovacuum_vacuum_scale_factor;    -- Default: 0.2 (20% de la tabla)
SHOW autovacuum_vacuum_insert_threshold; -- NUEVO PG 18: 1000 inserts

-- Ajuste para tablas grandes (recomendado)
ALTER TABLE logs SET (
    autovacuum_vacuum_scale_factor = 0.05,
    autovacuum_vacuum_threshold = 5000,
    autovacuum_vacuum_insert_threshold = 10000
);
```

Ejercicio Práctico Integrador (1 hora)

Objetivo: Identificar procesos, configurar memoria y crear tablespaces

Paso 1: Identificar procesos del sistema

bash

```
# Desde el terminal (Linux/Unix)
ps aux | grep postgres

# O desde SQL
psql -d postgres -c "SELECT pid, usename, application_name, backend_type FROM
pg_stat_activity;"
```

Paso 2: Ver y ajustar configuración de memoria

sql

```
-- Ver valores actuales
SELECT name, setting, unit, context
FROM pg_settings
WHERE name IN ('shared_buffers', 'work_mem', 'maintenance_work_mem');

-- Ajustar para la sesión actual (prueba)
SET work_mem = '64MB';
SET maintenance_work_mem = '512MB';

-- Verificar cambio
SHOW work_mem;
```

Paso 3: Crear y usar tablespaces

sql

```
-- Crear directorios (desde SQL con función especial)
\! sudo mkdir -p /tmp/pg18_ssd /tmp/pg18_hdd

-- Crear tablespaces
CREATE TABLESPACE ts_ssd LOCATION '/tmp/pg18_ssd';
CREATE TABLESPACE ts_hdd LOCATION '/tmp/pg18_hdd';

-- Crear tablas en diferentes tablespaces
CREATE TABLE usuarios_activos (id serial, nombre text) TABLESPACE ts_ssd;
CREATE TABLE logs_historicos (id serial, log text) TABLESPACE ts_hdd;

-- Ver ubicación de las tablas
SELECT relname, reltablespace,
       pg_tablespace_location(reltablespace) AS ubicacion
FROM pg_class
WHERE relname IN ('usuarios_activos', 'logs_historicos');
```

Paso 4: Simular MVCC y monitorear dead tuples

sql

```
-- Crear tabla de prueba
CREATE TABLE test_mvcc (id int primary key, valor text) TABLESPACE ts_ssd;

-- Insertar datos
INSERT INTO test_mvcc SELECT generate_series(1, 1000), 'original';

-- Ver estado inicial
```

```
SELECT n_live_tup, n_dead_tup, last_vacuum
FROM pg_stat_all_tables
WHERE relname = 'test_mvcc';

-- Actualizar muchas veces (crea dead tuples)
DO $$
BEGIN
    FOR i IN 1..100 LOOP
        UPDATE test_mvcc SET valor = 'actualizado_' || i WHERE id = i;
    END LOOP;
END $$;

-- Ver dead tuples acumulados
SELECT n_live_tup, n_dead_tup, n_mod_since_analyze
FROM pg_stat_all_tables
WHERE relname = 'test_mvcc';

-- Ejecutar VACUUM manual
VACUUM VERBOSE test_mvcc;

-- Verificar limpieza
SELECT n_live_tup, n_dead_tup, last_vacuum
FROM pg_stat_all_tables
WHERE relname = 'test_mvcc';
```

Paso 5: Analizar índices y planes de ejecución

sql

```
-- Crear índice con nueva funcionalidad de PG 18
CREATE INDEX CONCURRENTLY test_mvcc_idx ON test_mvcc (valor) WITH
(parallel_workers = 2);

-- Ver tamaño de índices
SELECT
    indexname,
    pg_size_pretty(pg_indexes_size('test_mvcc'::regclass)) AS indice_tamaño
FROM pg_indexes
WHERE tablename = 'test_mvcc';

-- Usar EXPLAIN para ver plan
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM test_mvcc WHERE valor LIKE 'actualizado_%';
```

Entregables del ejercicio:

1. Captura de la salida de `ps aux | grep postgres` identificando al menos 3 procesos diferentes.
2. Comandos SQL utilizados para modificar `work_mem` y verificar el cambio.
3. Lista de archivos creados en `/tmp/pg18_ssd/` después de crear la tabla.
4. Número de dead tuples generados en el paso 4 y tiempo que tardó VACUUM.
5. Plan de ejecución de la última consulta con el costo estimado.

Resumen del Módulo 1

Concepto	Enfoque tradicional	Novedad PostgreSQL 18
I/O	Síncrono	Async I/O con io_uring
Estadísticas	Globales	Por backend individual
Índices multicolumna	Requieren columna prefijo	Skip Scan automático
Monitoreo VACUUM	Solo contadores	Tiempo total invertido
Configuración memoria	Fija	`wal_buffers = auto`
Tablespaces	Migración con downtime	`pg_upgrade --swap`

Preparación para el Módulo 2

Temas a explorar antes de la siguiente clase:

- Familiarizarse con `pg_stat_io` y sus nuevas columnas (`read_bytes`, `write_bytes`).
- Practicar con `pg_stat_get_backend_io()` para una sesión específica.
- Entender la diferencia entre `buffers: shared hit` vs `buffers: shared read` en EXPLAIN.

Lectura recomendada:

- Documentación oficial de PostgreSQL 18: "Chapter 69. System Views"
- "PostgreSQL 18: What's New in MVCC" (blog oficial)

¿Listo para continuar? El Módulo 2 cubrirá monitoreo avanzado con las nuevas métricas de I/O asíncrono y diagnóstico de cuellos de botella en tiempo real.

desarrollame el modulo 2 para postgresql 18 desde basico hasta avanzado.

Leer 9 páginas web

Módulo 2: Monitoreo y Diagnóstico en PostgreSQL 18 (6 horas)

Objetivos del Módulo

- Comprender las nuevas capacidades de monitoreo introducidas en PostgreSQL 18.
- Utilizar las vistas de estadísticas por backend para diagnóstico granular.
- Dominar EXPLAIN con las nuevas opciones (WAL, SERIALIZE, MEMORY).
- Configurar y optimizar el subsistema de I/O asíncrono.

- Implementar una estrategia de monitoreo completa para entornos de producción.

2.1. Evolución del Subsistema de Estadísticas (1 hora)

Contexto: Limitaciones de Versiones Anteriores

Hasta PostgreSQL 17, el monitoreo tenía limitaciones significativas:

- Las estadísticas de I/O eran **agregadas** a nivel de todo el servidor (`pg_stat_io`).
- No era posible saber qué conexión específica estaba generando carga de disco.
- Las métricas se reportaban en **páginas (8KB)**, no en bytes, dificultando el cálculo.
- Para identificar consultas problemáticas en ejecución, los DBAs debían usar herramientas del sistema operativo como `iotop` y cruzar PIDs manualmente .

Novedades Arquitectónicas en PG 18

PostgreSQL 18 introduce un rediseño completo del sistema de estadísticas acumulativas :

Área de mejora	Descripción	Impacto práctico
Estadísticas por backend	Nuevas funciones `pg_stat_get_backend_io()` y `pg_stat_get_backend_wal()`	Identificar la consulta exacta que causa I/O alto
Reporte en bytes	`pg_stat_io` ahora muestra `read_bytes` y `write_bytes` en lugar de páginas	Cálculos directos sin multiplicar por 8192
API C para extensiones	Extensiones pueden usar el motor de estadísticas nativo	Desarrollo de herramientas de monitoreo más potentes
Retención de estadísticas	`pg_upgrade` preserva estadísticas del planificador	Sin "período frío" post-actualización
Límite de tabla hash	Aumentado de 1GB para soportar miles de tablas	Mejor para arquitecturas multi-tenant

Estructura de las Nuevas Funciones de Estadísticas

pg_stat_get_backend_io(pid integer)

Esta función retorna un conjunto de registros con las siguientes columnas :

sql

```
-- Estructura completa de pg_stat_get_backend_io()
-- OUT backend_type text
-- OUT object text (relation, temp_file, wal, etc.)
-- OUT context text (normal, vacuum, bulkread, bulkwrite)
-- OUT reads bigint
```

```
-- OUT read_time double precision (ms, si track_io_timing = on)
-- OUT writes bigint
-- OUT write_time double precision
-- OUT writebacks bigint
-- OUT writeback_time double precision
-- OUT extends bigint
-- OUT extend_time double precision
-- OUT op_bytes bigint
-- OUT hits bigint (aciertos en cache)
-- OUT evictions bigint
-- OUT reuses bigint
-- OUT fsyncs bigint
-- OUT fsync_time double precision
-- OUT stats_reset timestamptz
```

pg_stat_get_backend_wal(pid integer)

Proporciona estadísticas de Write-Ahead Log por proceso :

- `wal_records`: Número de registros WAL generados
- `wal_fpi`: Full Page Images escritas
- `wal_bytes`: Volumen total en bytes
- `wal_buffers_full`: Veces que el buffer WAL se llenó (útil para tuning)

Configuración Necesaria para Monitoreo Avanzado

conf

```
# postgresql.conf - Habilitar timing para estadísticas detalladas
track_io_timing = on          # Mide tiempos reales de I/O (requiere reinicio)
track_wal_io_timing = on      # Mide tiempos de WAL
track_activities = on         # Activo por defecto
track_counts = on             # Activo por defecto
track_functions = 'pl'        # Opcional: monitorear funciones PL/pgSQL
compute_query_id = auto       # Para identificar consultas en estadísticas
```

Advertencia importante: `track_io_timing` añade overhead por llamadas al reloj del sistema. En sistemas con muchas consultas cortas, el impacto puede ser significativo.
Use `pg_test_timing` para evaluar su plataforma antes de habilitarlo en producción .

2.2. Monitoreo de I/O por Backend (1.5 horas)

Diagnóstico en Tiempo Real

La combinación de `pg_stat_activity` con las nuevas funciones permite identificar instantáneamente qué conexiones están generando más carga de I/O .

Consulta Fundamental: Top de Consumidores de I/O

sql

```
-- Identificar los procesos con mayor actividad de lectura/escritura
SELECT
```

```
    a.pid,
    a.usename,
    a.application_name,
    a.client_addr,
    a.state,
    LEFT(a.query, 100) AS query_preview,
    -- Estadísticas de I/O
    COALESCE(SUM(io.reads), 0) AS reads,
    COALESCE(pg_size_pretty(SUM(io.read_bytes)), '0 bytes') AS read_bytes,
    COALESCE(SUM(io.writes), 0) AS writes,
    COALESCE(pg_size_pretty(SUM(io.write_bytes)), '0 bytes') AS write_bytes,
    COALESCE(ROUND(SUM(io.read_time + io.write_time) / 1000, 2), 0) AS
io_seconds,
    -- Estadísticas de WAL
    COALESCE(pg_size_pretty(SUM(wal.wal_bytes)), '0 bytes') AS wal_generated,
    COALESCE(SUM(wal.wal_records), 0) AS wal_records,
    COALESCE(SUM(wal.wal_buffers_full), 0) AS wal_buffers_full
FROM pg_stat_activity a
LEFT JOIN pg_stat_get_backend_io(a.pid) io ON true
LEFT JOIN pg_stat_get_backend_wal(a.pid) wal ON true
WHERE a.backend_type = 'client backend'  -- Solo conexiones de clientes
  AND a.state != 'idle'                   -- Consultas activas
GROUP BY a.pid, a.usename, a.application_name, a.client_addr, a.state, a.query
ORDER BY (SUM(io.read_bytes) + SUM(io.write_bytes)) DESC
LIMIT 10;
```

Análisis Detallado de un Proceso Específico

sql

```
-- Ver desglose por tipo de objeto (tablas vs archivos temporales)
SELECT
    object,
    context,
    reads,
    pg_size_pretty(read_bytes) AS read_bytes,
    read_time,
    writes,
    pg_size_pretty(write_bytes) AS write_bytes,
    write_time,
    extends,
    hits,
    evictions
FROM pg_stat_get_backend_io(12345)  -- Reemplazar con PID real
WHERE read_bytes > 0 OR write_bytes > 0
ORDER BY read_bytes DESC;
```

Interpretación de Resultados

Observación	Posible Causa	Acción Recomendada
`reads` alto, `hits` bajo	Cache miss, tabla no cabe en shared_buffers	Aumentar shared_buffers o particionar tabla
`read_time` alto, pocas lecturas	Discos lentos o I/O contienda	Migrar a SSD, separar tablespaces
`extends` alto	Crecimiento frecuente de la tabla	Ajustar `autovacuum_vacuum_scale_factor`
`evictions` alto	Buffer pressure, muchas	Aumentar shared_buffers o

Observación	Posible Causa	Acción Recomendada
	tablas compitiendo	reducir work_mem
`wal_buffers_full` > 0	Buffer WAL insuficiente	Aumentar `wal_buffers` (mínimo 32MB)

Monitoreo de Sesiones con Connection Pooling

Advertencia crítica: En entornos con PgBouncer o connection pooling, las estadísticas se acumulan durante toda la vida de la conexión backend. Una misma conexión física puede haber ejecutado múltiples consultas de diferentes clientes .

Estrategia para entornos con pooling:

sql

```
-- 1. Primero, identificar conexiones con I/O acumulado alto
-- 2. Luego, verificar si la query actual es responsable o es residual
-- 3. Considerar resetear estadísticas periódicamente

-- Resetear estadísticas de un backend específico (nuevo en PG 18)
SELECT pg_stat_reset_backend_stats(12345);
```

2.3. EXPLAIN en PostgreSQL 18: Opciones Avanzadas (1.5 horas)

Nuevas Opciones en PG 18

PostgreSQL 18 introduce varias opciones nuevas para EXPLAIN que permiten un análisis mucho más profundo :

Opción	Descripción	Disponible desde
WAL	Muestra generación de WAL (registros, FPI, bytes)	PG 18
SERIALIZE	Mide costo de serialización a texto/binario	PG 18
MEMORY	Muestra consumo de memoria durante planificación	PG 18
BUFFERS	Ahora incluido automáticamente con ANALYZE	PG 18
GENERIC_PLAN	Genera plan genérico sin valores específicos	PG 18

Sintaxis Completa de EXPLAIN en PG 18

sql

```
EXPLAIN [ (
    ANALYZE [boolean] |
    VERBOSE [boolean] |
```

```
    COSTS [boolean] |
    SETTINGS [boolean] |
    GENERIC_PLAN [boolean] |
    BUFFERS [boolean] |
    SERIALIZE { NONE | TEXT | BINARY } |
    WAL [boolean] |
    TIMING [boolean] |
    SUMMARY [boolean] |
    MEMORY [boolean] |
    FORMAT { TEXT | XML | JSON | YAML }
) ] statement;
```

Ejemplos Prácticos por Caso de Uso

Caso 1: Análisis Completo de I/O con BUFFERS

sql

```sql
-- Analizar consulta con estadísticas completas de buffers
EXPLAIN (ANALYZE, BUFFERS, TIMING ON)
SELECT * FROM orders
WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31'
  AND status = 'COMPLETED';

-- Interpretación de salida:
-- buffers: shared hit=1500 (encontrados en cache)
--          shared read=500 (tuvieron que leerse de disco)
--          temp read/write=200 (uso de disco para ordenamiento)
```

Caso 2: Medición de WAL Generation (Nuevo en PG 18)

sql

```sql
-- Medir cuánto WAL genera una operación de actualización masiva
EXPLAIN (ANALYZE, WAL, BUFFERS)
UPDATE orders SET processed = true
WHERE order_date < '2025-01-01';

-- Salida esperada:
-- WAL: records=1523 fpi=45 bytes=1892352
--      fpi (Full Page Images) indica páginas completas escritas por checkpoint
```

Caso 3: Serialización y Envío al Cliente

sql

```sql
-- Medir el costo de convertir datos a formato para el cliente
EXPLAIN (ANALYZE, SERIALIZE TEXT, BUFFERS)
SELECT jsonb_agg(row_to_json(orders)) FROM orders WHERE customer_id = 12345;

-- Útil para consultas que retornan grandes volúmenes de JSON/JSONB
```

Caso 4: Memoria Usada en Planificación

sql

```sql
-- Ver cuánta memoria consumió el planificador
EXPLAIN (MEMORY, FORMAT JSON)
SELECT * FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN order_items i ON o.id = i.order_id
WHERE c.created_at > '2026-01-01';
```

Mejores Prácticas con EXPLAIN en Producción

sql

```
-- 1. Para consultas DDL peligrosas, usar transacción con rollback
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
DELETE FROM logs WHERE created_at < '2025-01-01';
ROLLBACK;

-- 2. Guardar planes de referencia para comparación
\copy (EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ...) TO 'plan_ref.json';

-- 3. Usar formato JSON para herramientas automatizadas
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ...;
```

2.4. Monitoreo del Subsistema de I/O Asíncrono (AIO) (1 hora)

Parámetros de Configuración de AIO en PG 18

PostgreSQL 18 introduce nuevos GUCs para controlar el comportamiento del I/O asíncrono :

Parámetro	Descripción	Valor por defecto	Rango
`io_method`	Método de I/O (sync, worker, io_uring)	worker	-
`io_workers`	Workers para I/O asíncrono	3	1-32
`io_combine_limit`	Bloques adyacentes a combinar (unidad: 8KB)	16 (128KB)	1-128
`io_max_combine_limit`	Límite global de combinación	16	restart required
`io_max_concurrency`	Solicitudes I/O concurrentes por proceso	-1 (auto)	-1 a 1024
`effective_io_concurrency`	Para consultas (bitmap scans)	16	-
`maintenance_io_concurrency`	Para VACUUM y mantenimiento	16	-

Verificación y Monitoreo de AIO

sql

```
-- Ver configuración actual
SELECT name, setting, unit, context
FROM pg_settings
WHERE name LIKE '%io_%' OR name LIKE '%concurrency%';

-- Ver estadísticas de I/O globales (ahora en bytes)
SELECT
```

```
    backend_type,
    object,
    context,
    reads,
    pg_size_pretty(read_bytes) AS read_bytes,
    read_time,
    writes,
    pg_size_pretty(write_bytes) AS write_bytes,
    write_time,
    extends,
    hits
FROM pg_stat_io
WHERE reads > 0 OR writes > 0
ORDER BY read_bytes DESC
LIMIT 10;
```

Evaluación de Rendimiento de AIO

sql

```
-- Crear tabla de prueba
CREATE TABLE test_aio AS
SELECT generate_series(1, 5000000) AS id,
       md5(random()::text) AS data;

-- Prueba con método sync vs async
-- Método 1: sync (reiniciar PostgreSQL con io_method='sync')
EXPLAIN (ANALYZE, BUFFERS, TIMING ON)
SELECT count(*) FROM test_aio WHERE data LIKE '%abc%';

-- Método 2: async con worker
-- Reiniciar con io_method='worker', luego ejecutar misma consulta

-- Comparar tiempos de ejecución y lecturas
```

Monitoreo de Workers de I/O

sql

```
-- Ver workers de I/O activos (nuevo en PG 18)
SELECT pid, backend_type, state, query
FROM pg_stat_activity
WHERE backend_type = 'I/O worker';

-- Estadísticas por worker
SELECT
    io.backend_type,
    io.object,
    io.reads,
    io.read_time,
    io.writebacks
FROM pg_stat_io io
WHERE io.backend_type = 'I/O worker';
```

2.5. Estadísticas de Mantenimiento: VACUUM y ANALYZE (1 hora)

Nuevas Columnas en pg_stat_all_tables

PostgreSQL 18 añade información temporal detallada sobre operaciones de mantenimiento :

sql

```
-- Nuevas columnas en pg_stat_all_tables
SELECT
    schemaname,
    tablename,
    -- Información de VACUUM
    last_vacuum,
    last_autovacuum,
    vacuum_count,
    autovacuum_count,
    vacuum_time,        -- NUEVO: tiempo total invertido en VACUUM manual
    autovacuum_time,    -- NUEVO: tiempo total en autovacuum
    -- Información de ANALYZE
    last_analyze,
    last_autoanalyze,
    analyze_count,
    autoanalyze_count,
    analyze_time,       -- NUEVO: tiempo total en ANALYZE manual
    autoanalyze_time,   -- NUEVO: tiempo total en autoanalyze
    -- Tuplas
    n_live_tup,
    n_dead_tup,
    n_mod_since_analyze
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC;
```

track_cost_delay_timing (Nuevo GUC)

Este parámetro controla la recolección de estadísticas de delay en VACUUM y ANALYZE :

conf

```
# postgresql.conf
track_cost_delay_timing = on   # Recolecta tiempos de sleep por cost_delay
```

Importante: Al igual que `track_io_timing`, este parámetro añade overhead. Verificar con `pg_test_timing` antes de habilitar en producción.

Monitoreo de Autovacuum en PG 18

sql

```
-- Progreso de VACUUM con más detalles
SELECT
    pid,
    datname,
    relname,
    phase,
    heap_blks_total,
    heap_blks_scanned,
    heap_blks_vacuumed,
```

```
    index_vacuum_count,
    max_dead_tuples,
    num_dead_tuples
FROM pg_stat_progress_vacuum;

-- Identificar tablas que nunca son vacunadas
SELECT
    schemaname,
    tablename,
    n_dead_tup,
    last_autovacuum,
    autovacuum_count,
    age(relfrozenxid) AS xid_age
FROM pg_stat_user_tables t
JOIN pg_class c ON t.relname = c.relname
WHERE last_autovacuum IS NULL
   OR autovacuum_count = 0
ORDER BY n_dead_tup DESC;
```

Ejercicio Práctico Integrador (1 hora)

Objetivo: Implementar un Dashboard de Monitoreo en Tiempo Real

Paso 1: Configurar el Entorno

sql

```
-- Crear extensión para estadísticas de consultas (si no existe)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Verificar configuración de monitoreo
SHOW track_io_timing;
SHOW track_wal_io_timing;
SHOW pg_stat_statements.track;

-- Si es necesario, habilitar
ALTER SYSTEM SET track_io_timing = on;
ALTER SYSTEM SET track_wal_io_timing = on;
SELECT pg_reload_conf();
```

Paso 2: Crear Funciones de Monitoreo

sql

```
-- Función para obtener top de consumidores de I/O
CREATE OR REPLACE FUNCTION get_top_io_consumers(limit_n int DEFAULT 10)
RETURNS TABLE (
    pid int,
    usuario text,
    aplicacion text,
    consulta text,
    lectura_mb numeric,
    escritura_mb numeric,
    tiempo_io_seg numeric,
    wal_mb numeric
) AS $$
BEGIN
    RETURN QUERY
    SELECT
```

```
        a.pid,
        a.usename,
        COALESCE(a.application_name, 'unknown'),
        LEFT(a.query, 200),
        ROUND(COALESCE(SUM(io.read_bytes), 0) / 1048576.0, 2),
        ROUND(COALESCE(SUM(io.write_bytes), 0) / 1048576.0, 2),
        ROUND(COALESCE(SUM(io.read_time + io.write_time), 0) / 1000.0, 2),
        ROUND(COALESCE(SUM(wal.wal_bytes), 0) / 1048576.0, 2)
    FROM pg_stat_activity a
    LEFT JOIN pg_stat_get_backend_io(a.pid) io ON true
    LEFT JOIN pg_stat_get_backend_wal(a.pid) wal ON true
    WHERE a.backend_type = 'client backend'
      AND a.state = 'active'
    GROUP BY a.pid, a.usename, a.application_name, a.query
    ORDER BY (SUM(io.read_bytes) + SUM(io.write_bytes)) DESC
    LIMIT limit_n;
END;
$$ LANGUAGE plpgsql;

-- Uso:
SELECT * FROM get_top_io_consumers(5);
```

Paso 3: Monitorear Checkpoints y WAL

sql

```
-- Estadísticas de checkpointer (nuevo en PG 18)
SELECT
    checkpoints_timed,
    checkpoints_req,
    checkpoint_write_time,
    checkpoint_sync_time,
    buffers_checkpoint,
    num_done,           -- NUEVO: número de checkpoints completados
    slru_written        -- NUEVO: buffers SLRU escritos
FROM pg_stat_checkpointer;

-- Monitoreo de generación WAL
SELECT
    pg_current_wal_lsn() AS current_lsn,
    pg_wal_lsn_diff(pg_current_wal_lsn(), '0/0') AS wal_bytes_total,
    pg_current_wal_insert_lsn() AS insert_lsn;
```

Paso 4: Análisis de Bloat y Tuplas Muertas

sql

```
-- Identificar tablas con alto bloat
SELECT
    schemaname,
    tablename,
    n_live_tup,
    n_dead_tup,
    ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS
dead_ratio,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS
total_size,
    last_autovacuum,
    autovacuum_time AS vacuum_time_ms
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC
LIMIT 10;
```

Paso 5: Simular Carga y Analizar con EXPLAIN Avanzado

sql

```
-- 1. Crear datos de prueba
CREATE TABLE orders_test (
    id serial PRIMARY KEY,
    customer_id int,
    order_date date,
    total numeric(10,2),
    status text,
    details jsonb
);

INSERT INTO orders_test (customer_id, order_date, total, status, details)
SELECT
    (random() * 10000)::int,
    date '2025-01-01' + (random() * 365)::int,
    (random() * 1000)::numeric(10,2),
    CASE WHEN random() > 0.7 THEN 'COMPLETED' ELSE 'PENDING' END,
    jsonb_build_object('items', (random() * 10)::int, 'notes',
md5(random()::text))
FROM generate_series(1, 500000);

-- 2. Ejecutar análisis completo
EXPLAIN (ANALYZE, BUFFERS, WAL, SERIALIZE TEXT, FORMAT JSON)
SELECT
    customer_id,
    COUNT(*) as orders,
    SUM(total) as total_spent
FROM orders_test
WHERE order_date BETWEEN '2025-06-01' AND '2025-12-31'
  AND status = 'COMPLETED'
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 100;

-- 3. Verificar estadísticas de I/O después de la consulta
SELECT * FROM get_top_io_consumers(3);
```

Paso 6: Interpretación y Reporte

Preguntas a responder en el reporte del ejercicio:

1. ¿Qué PID generó más I/O durante la consulta?
2. ¿Cuántos buffers fueron leídos del disco vs encontrados en cache?
3. ¿Cuánto WAL se generó (en bytes)?
4. ¿Cuál fue el tiempo de serialización para enviar los resultados?
5. ¿El planificador usó un Seq Scan o un Index Scan? ¿Por qué?

Resumen del Módulo 2

Componente	Versiones anteriores	PostgreSQL 18
I/O por sesión	No disponible	`pg_stat_get_backend_io()`
Unidades de I/O	Páginas (8KB)	Bytes
EXPLAIN WAL	No disponible	`EXPLAIN (WAL)`
EXPLAIN SERIALIZE	No disponible	`EXPLAIN (SERIALIZE)`
Tiempos de VACUUM	Solo contadores	`vacuum_time`, `autovacuum_time`
Estadísticas post-upgrade	Pérdida total	Retención con `pg_upgrade`
Checkpoints completados	No disponible	`num_done` en `pg_stat_checkpointer`

Preparación para el Módulo 3

Temas a explorar:

- Familiarizarse con `pg_stat_statements` y su nuevo agrupamiento de consultas con `IN`
- Practicar con las nuevas columnas de conflictos en replicación lógica: `confl_insert_exists`, `confl_update_missing`
- Entender el impacto de `track_cost_delay_timing` en sistemas de alta concurrencia

Lectura recomendada:

- Documentación oficial: "Chapter 28. Monitoring Database Activity" (PG 18)
- Blog: "Per-backend IO stats in PostgreSQL 18" de Redgate

¿Listo para continuar? El Módulo 3 cubrirá optimización avanzada de consultas, índices y el nuevo optimizador de PG 18.

ok. Desarrollame el module 3 para postgresql 18

Leer 8 páginas web

Módulo 3: Optimización Avanzada en PostgreSQL 18 (6 horas)

Objetivos del Módulo

- Dominar las nuevas técnicas de optimización introducidas en PostgreSQL 18.

- Implementar estrategias de indexación avanzadas aprovechando UUIDv7 y Skip Scan.
- Utilizar estadísticas multivariantes para mejorar estimaciones del planificador.
- Optimizar consultas con las nuevas capacidades de JOIN y planificación.
- Aplicar técnicas de particionamiento y optimización de DDL.

3.1. Nuevas Técnicas de Indexación en PG 18 (1.5 horas)

Skip Scan: Revolución en Índices Multicolumna

PostgreSQL 18 introduce una optimización fundamental en la forma en que utiliza índices B-tree multicolumna .

El problema histórico: Cuando tienes un índice `(status, created_at)`, una consulta como `WHERE created_at > '2026-01-01'` no podía usar el índice porque la columna `status` no estaba filtrada. El planificador asumía que necesitaba recorrer todos los valores de `status`.

La solución en PG 18: El nuevo **Skip Scan** permite al planificador "saltarse" la primera columna del índice si tiene pocos valores distintos .

sql

```
-- Crear tabla de ejemplo
CREATE TABLE pedidos (
    id SERIAL PRIMARY KEY,
    status TEXT,
    created_at TIMESTAMPTZ,
    total DECIMAL(10,2),
    customer_id INT
);

-- Insertar datos (100,000 registros, status tiene solo 5 valores)
INSERT INTO pedidos (status, created_at, total, customer_id)
SELECT
    CASE (random() * 4)::INT
        WHEN 0 THEN 'PENDING'
        WHEN 1 THEN 'PROCESSING'
        WHEN 2 THEN 'COMPLETED'
        WHEN 3 THEN 'SHIPPED'
        ELSE 'CANCELLED'
    END,
    NOW() - (random() * INTERVAL '365 days'),
    (random() * 1000)::DECIMAL(10,2),
    (random() * 10000)::INT
FROM generate_series(1, 100000);

-- Crear índice multicolumna
CREATE INDEX idx_pedidos_status_fecha ON pedidos (status, created_at);

-- ANALYZE para actualizar estadísticas
ANALYZE pedidos;
```

Verificando Skip Scan en acción:

sql

```
-- Esta consulta ahora puede usar Skip Scan
EXPLAIN (ANALYZE, BUFFERS, TIMING ON)
SELECT * FROM pedidos
WHERE created_at > NOW() - INTERVAL '30 days'
ORDER BY created_at;

-- El plan mostrará algo similar a:
-- "Index Skip Scan using idx_pedidos_status_fecha"
```

Cuándo funciona Skip Scan :

- La columna omitida tiene **pocos valores distintos** (recomendado < 100)
- El índice es B-tree
- La consulta ordena o filtra por la segunda columna

Limitaciones a conocer:

- No funciona con `LIKE` en la columna omitida
- La estimación de costo puede ser conservadora en tablas muy grandes
- Monitorear con `pg_stats` la cardinalidad de la primera columna

UUIDv7: El Fin de la Fragmentación de Índices

Uno de los problemas más comunes en PostgreSQL ha sido el uso de UUIDv4 como clave primaria. La aleatoriedad de UUIDv4 causa :

- **Page splits frecuentes**: Cada inserción puede ir a una página diferente
- **Fragmentación del índice**: El índice B-tree pierde localidad
- **Cache ineficiente**: Las inserciones no aprovechan el caché

UUIDv7 resuelve esto incorporando un timestamp Unix en los primeros 48 bits, haciendo que los UUIDs sean **monotónicos** (ordenados cronológicamente) .

sql

```
-- Generar UUIDv7 (nuevo en PG 18)
SELECT uuidv7();
-- Resultado ejemplo: 01980de8-ad3d-715c-b739-faf2bb1a7aad

-- Extraer timestamp de un UUIDv7
SELECT uuid_extract_timestamp(uuidv7());
-- Resultado: 2025-12-15 10:30:45.123+00

-- Verificar la versión
SELECT uuid_extract_version(uuidv7());  -- Retorna 7
```

Creando tablas con UUIDv7 como PK:

sql

```
-- Tabla óptima para inserción masiva
CREATE TABLE logs_auditoria (
    id UUID PRIMARY KEY DEFAULT uuidv7(),
```

```
    evento TEXT NOT NULL,
    usuario TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    metadata JSONB
);

-- Insertar datos (las inserciones serán secuenciales)
INSERT INTO logs_auditoria (evento, usuario, metadata)
SELECT
    'ACCESO_USUARIO',
    'user_' || i,
    jsonb_build_object('ip', '192.168.1.' || (i % 255), 'user_agent',
'Mozilla/5.0')
FROM generate_series(1, 10000) i;
```

Comparativa de rendimiento: UUIDv4 vs UUIDv7

sql

```
-- Crear tablas de prueba
CREATE TABLE test_uuid_v4 (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), data
TEXT);
CREATE TABLE test_uuid_v7 (id UUID PRIMARY KEY DEFAULT uuidv7(), data TEXT);

-- Insertar 100,000 registros en cada una
INSERT INTO test_uuid_v4 (data) SELECT md5(i::TEXT) FROM generate_series(1,
100000) i;
INSERT INTO test_uuid_v7 (data) SELECT md5(i::TEXT) FROM generate_series(1,
100000) i;

-- Analizar fragmentación del índice
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan,
    idx_tup_read,
    pg_size_pretty(pg_relation_size(indexname::regclass)) AS index_size
FROM pg_stat_user_indexes
WHERE tablename IN ('test_uuid_v4', 'test_uuid_v7');

-- Verificar el bloat del índice (más alto en UUIDv4)
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS
total_size,
    round(100 * pg_stat_get_live_tuples(tablename::regclass) /
          nullif(pg_stat_get_dead_tuples(tablename::regclass) +
                 pg_stat_get_live_tuples(tablename::regclass), 0), 2) AS
live_ratio
FROM pg_stat_user_tables
WHERE tablename IN ('test_uuid_v4', 'test_uuid_v7');
```

Migración de UUIDv4 a UUIDv7 en producción :

sql

```
-- 1. Agregar nueva columna con UUIDv7 como default
ALTER TABLE orders ADD COLUMN id_new UUID DEFAULT uuidv7();

-- 2. Poblar columna existente (puede hacerse en lotes)
UPDATE orders SET id_new = uuidv7() WHERE id_new IS NULL;
```

```
-- 3. Cambiar el default de la columna original
ALTER TABLE orders ALTER COLUMN id SET DEFAULT uuidv7();

-- 4. Para nuevas tablas, usar uuidv7() directamente
```

Índices GIN Paralelos y Nuevas Capacidades

PostgreSQL 18 añade **creación paralela de índices GIN**, igualando la funcionalidad que ya existía para B-tree y BRIN .

sql

```
-- Crear índice GIN en paralelo (para JSONB o texto completo)
CREATE INDEX CONCURRENTLY idx_productos_datos_gin ON productos USING gin
(datos_jsonb)
WITH (parallel_workers = 4);

-- Verificar workers utilizados
EXPLAIN (ANALYZE, BUFFERS)
CREATE INDEX idx_test_gin ON test_table USING gin (jsonb_column);
```

Mejoras en Full-Text Search:

- Nueva collation `PG_UNICODE_FAST` para comparaciones más rápidas
- Transformación de mayúsculas/minúsculas optimizada

sql

```
-- Usar nueva collation para búsquedas
CREATE INDEX idx_documentos_texto ON documentos USING gin
(to_tsvector('spanish', contenido))
WITH (parallel_workers = 2);

-- Búsqueda optimizada
SELECT * FROM documentos
WHERE to_tsvector('spanish', contenido) @@ to_tsquery('spanish', 'optimización &
base & datos');
```

3.2. Optimización del Planificador y Estadísticas (1.5 horas)

Estadísticas Multivariantes: Correlaciones y Dependencias

El planificador de PostgreSQL asume independencia entre columnas a menos que se le indique lo contrario. Esto puede causar estimaciones terriblemente incorrectas .

Ejemplo del problema:

sql

```
-- Tabla con correlación perfecta entre columnas
CREATE TABLE t (a INT, b INT);
INSERT INTO t SELECT i % 100, i % 100 FROM generate_series(1, 10000) s(i);
ANALYZE t;

-- Sin estadísticas multivariantes, la estimación es catastrófica
EXPLAIN (ANALYZE, TIMING OFF) SELECT * FROM t WHERE a = 1 AND b = 1;
-- El planificador estima 1 fila, pero hay 100 filas reales
```

Solución: CREATE STATISTICS

PostgreSQL permite crear estadísticas que capturan dependencias funcionales, correlaciones y listas de valores más comunes multivariantes.

sql

```
-- 1. Estadísticas de dependencias funcionales
CREATE STATISTICS stts_dependencies (dependencies) ON a, b FROM t;
ANALYZE t;

-- Ahora la estimación es precisa
EXPLAIN (ANALYZE, TIMING OFF) SELECT * FROM t WHERE a = 1 AND b = 1;
-- rows estimadas: 100 (correcto)

-- 2. Estadísticas de ndistinct (valores únicos)
CREATE STATISTICS stts_ndistinct (ndistinct) ON a, b FROM t;
ANALYZE t;

-- Mejora estimaciones de GROUP BY
EXPLAIN (ANALYZE, TIMING OFF) SELECT COUNT(*) FROM t GROUP BY a, b;

-- 3. Lista MCV (Most Common Values) multivariante
CREATE STATISTICS stts_mcv (mcv) ON a, b FROM t;
ANALYZE t;

-- Inspeccionar la lista MCV
SELECT m.* FROM pg_statistic_ext
JOIN pg_statistic_ext_data ON (oid = stxoid),
LATERAL pg_mcv_list_items(stxdmcv) m
WHERE stxname = 'stts_mcv';
```

Tipos de estadísticas multivariantes :

Tipo	Cuándo usarlo	Overhead
`dependencies`	Columnas funcionalmente dependientes	Bajo
`ndistinct`	GROUP BY con múltiples columnas	Medio
`mcv`	Valores específicos con correlación fuerte	Alto

Ejemplo práctico en sistema real:

sql

```
-- Escenario: tabla de ventas con correlación producto-categoría
CREATE TABLE ventas (
    id SERIAL PRIMARY KEY,
    producto_id INT,
    categoria_id INT,
    fecha DATE,
    monto DECIMAL(10,2)
);

-- Las categorías determinan productos (dependencia funcional)
-- Pero el planificador no lo sabe

-- Crear estadísticas
CREATE STATISTICS ventas_prod_cat (dependencies, ndistinct)
ON producto_id, categoria_id FROM ventas;
```

```
ANALYZE ventas;

-- Las consultas con ambos filtros ahora estiman correctamente
EXPLAIN SELECT * FROM ventas
WHERE producto_id = 12345 AND categoria_id = 5;
```

Retención de Estadísticas Post-Upgrade

Una mejora crítica en PostgreSQL 18 es la **preservación de estadísticas durante pg_upgrade** . Anteriormente, después de una actualización mayor, todas las estadísticas se perdían y el planificador operaba "a ciegas" hasta que ANALYZE se ejecutaba.

Impacto de la mejora:

- No más "período frío" de rendimiento degradado post-actualización
- ANALYZE puede ejecutarse en segundo plano sin presión de tiempo
- Las consultas críticas mantienen planes óptimos inmediatamente

Verificación de estadísticas retenidas:

sql

```
-- Ver cuándo se actualizaron las estadísticas
SELECT
    schemaname,
    tablename,
    last_analyze,
    last_autoanalyze,
    n_mod_since_analyze
FROM pg_stat_user_tables
WHERE last_analyze IS NOT NULL;

-- Después de pg_upgrade a PG 18, estas fechas se mantienen
```

Optimización de JOINs en PG 18

PostgreSQL 18 incluye mejoras significativas en la planificación y ejecución de joins :

1. Hash Joins más rápidos:

- Mejor uso de caché L2/L3
- Algoritmo optimizado para tablas con skew

2. Merge Joins con ordenamiento incremental:

- Puede usar ordenamientos parciales para reducir trabajo
- Beneficioso cuando una tabla ya está ordenada por una columna del JOIN

3. OR-expansion:
Las consultas con condiciones `OR` ahora pueden usar índices de manera más efectiva .

sql

```
-- Antes de PG 18: difícil de optimizar
SELECT * FROM pedidos
WHERE status = 'PENDING' OR created_at > '2026-01-01';
```

```
-- En PG 18: puede convertir a UNION y usar índices separados
-- Equivalente optimizado (el planificador lo hace automáticamente)
SELECT * FROM pedidos WHERE status = 'PENDING'
UNION ALL
SELECT * FROM pedidos WHERE created_at > '2026-01-01' AND status != 'PENDING';
```

Verificando mejoras en JOINs:

sql

```
-- Tablas de prueba
CREATE TABLE clientes (id SERIAL PRIMARY KEY, nombre TEXT, ciudad TEXT);
CREATE TABLE compras (id SERIAL PRIMARY KEY, cliente_id INT REFERENCES
clientes(id), monto DECIMAL(10,2));

-- Insertar datos
INSERT INTO clientes SELECT i, 'Cliente_' || i, 'Ciudad_' || (i % 50) FROM
generate_series(1, 10000) i;
INSERT INTO compras SELECT i, (random() * 9999 + 1)::INT, random() * 1000 FROM
generate_series(1, 100000) i;

ANALYZE;

-- Analizar plan de Hash Join
EXPLAIN (ANALYZE, BUFFERS, TIMING ON)
SELECT c.nombre, SUM(co.monto) as total
FROM clientes c
JOIN compras co ON c.id = co.cliente_id
WHERE c.ciudad = 'Ciudad_25'
GROUP BY c.nombre;
```

3.3. Optimización de DDL y Operaciones de Mantenimiento (1.5 horas)

Columnas Generadas Virtuales

PostgreSQL 18 hace que las **columnas generadas virtuales** sean el valor predeterminado, en lugar de stored .

Diferencia clave:

- **VIRTUAL** (nuevo default): Calcula el valor en tiempo de lectura, **no ocupa espacio en disco**
- **STORED**: Calcula y almacena el valor, ocupa espacio pero puede indexarse

sql

```
-- Columna virtual (default en PG 18)
CREATE TABLE productos (
    id SERIAL PRIMARY KEY,
    precio_base DECIMAL(10,2) NOT NULL,
    iva DECIMAL(4,2) DEFAULT 21.00,
    descuento DECIMAL(4,2) DEFAULT 0.00,

    -- Virtual: se calcula al leer, no ocupa espacio
    precio_final DECIMAL(10,2) GENERATED ALWAYS AS (
        precio_base * (1 + iva/100) * (1 - descuento/100)
```

```
    ) VIRTUAL,  -- VIRTUAL es opcional, es el default

    -- Stored: ocupa espacio, pero se puede indexar
    precio_con_iva DECIMAL(10,2) GENERATED ALWAYS AS (
        precio_base * (1 + iva/100)
    ) STORED
);

-- Índice solo posible en columnas STORED
CREATE INDEX idx_productos_precio_iva ON productos (precio_con_iva);
```

Cuándo usar VIRTUAL vs STORED :

Escenario	Recomendación	Razón
Cálculos simples en consultas frecuentes	VIRTUAL	Ahorra espacio, CPU barato
La columna se usa en WHERE/JOIN	STORED + índice	Necesita indexación
Replicación lógica	STORED	VIRTUAL no se replica
La tabla tiene millones de filas	VIRTUAL	Ahorro significativo de espacio

NOT NULL sin Bloqueos con NOT VALID

Una de las operaciones más problemáticas en bases de datos grandes es agregar una restricción `NOT NULL` a una columna existente. PostgreSQL 18 introduce una sintaxis que permite hacerlo **sin bloquear la tabla** .

sql

```
-- Método tradicional (bloquea la tabla para escritura)
ALTER TABLE usuarios ALTER COLUMN email SET NOT NULL;  -- Bloquea hasta validar

-- Nuevo método en PG 18 (instantáneo, sin bloqueo)
ALTER TABLE usuarios ALTER COLUMN email SET NOT NULL NOT VALID;

-- Luego, en un momento de baja carga, validar la restricción
ALTER TABLE usuarios VALIDATE CONSTRAINT usuarios_email_not_null;

-- La validación hace un escaneo completo pero con lock SHARE UPDATE EXCLUSIVE
-- (permite lecturas, bloquea escrituras pero por menos tiempo)
```

Ventajas del nuevo enfoque:

- La restricción es visible inmediatamente para el planificador
- Nuevas inserciones/actualizaciones validan la restricción
- La validación final puede ejecutarse en mantenimiento programado

Mejoras en Particionamiento y Foreign Keys

Foreign Keys en tablas particionadas:

PostgreSQL 18 ahora soporta `NOT VALID` en foreign keys de tablas particionadas .

sql

```
-- Crear tabla particionada
CREATE TABLE eventos (
    id SERIAL,
    fecha DATE NOT NULL,
    datos TEXT
) PARTITION BY RANGE (fecha);

-- Crear particiones
CREATE TABLE eventos_2025 PARTITION OF eventos FOR VALUES FROM ('2025-01-01') TO
('2026-01-01');
CREATE TABLE eventos_2026 PARTITION OF eventos FOR VALUES FROM ('2026-01-01') TO
('2027-01-01');

-- Agregar FK sin bloqueo (nuevo)
ALTER TABLE eventos
ADD CONSTRAINT fk_eventos_categorias
FOREIGN KEY (categoria_id) REFERENCES categorias(id)
NOT VALID;

-- Validar después
ALTER TABLE eventos VALIDATE CONSTRAINT fk_eventos_categorias;
```

VACUUM más agresivo en PG 18:

PostgreSQL 18 congela proactivamente más páginas durante VACUUM normales, reduciendo la sobrecarga de transacciones .

sql

```
-- Monitorear edad de transacciones
SELECT
    datname,
    age(datfrozenxid) AS xid_age,
    pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database
ORDER BY age(datfrozenxid) DESC;

-- Configuración para tablas muy activas
ALTER TABLE logs_transacciones SET (
    autovacuum_vacuum_scale_factor = 0.0,
    autovacuum_vacuum_threshold = 5000,
    autovacuum_freeze_min_age = 1000000,
    autovacuum_freeze_table_age = 10000000
);
```

pg_upgrade --swap: Migraciones sin Downtime

Una mejora operacional crítica: la opción `--swap` en pg_upgrade .

bash

```
# Método tradicional (con downtime)
pg_upgrade --old-datadir /data/pg17 --new-datadir /data/pg18 --old-bindir
/usr/pg17/bin --new-bindir /usr/pg18/bin

# Nuevo método con --swap (downtime reducido al mínimo)
pg_upgrade --swap --old-datadir /data/pg17 --new-datadir /data/pg18 --old-bindir
/usr/pg17/bin --new-bindir /usr/pg18/bin

# Verificación paralela de objetos
pg_upgrade --jobs=4 --check
```

Flujo de trabajo recomendado:

1. Ejecutar `pg_upgrade --check --jobs=4` (sin downtime)
2. Resolver cualquier incompatibilidad
3. Ejecutar `pg_upgrade --swap` (downtime de segundos)
4. Los directorios se intercambian, no se copian

3.4. SQL Avanzado para Optimización (1.5 horas)

RETURNING con OLD/NEW

Una adición muy esperada: acceder a los valores antiguos y nuevos en la cláusula `RETURNING` .

sql

```
-- Auditoría simple con OLD y NEW
UPDATE empleados
SET salario = salario * 1.10,
    updated_at = NOW()
WHERE departamento = 'VENTAS'
RETURNING
    OLD.salario AS salario_anterior,
    NEW.salario AS salario_nuevo,
    OLD.nombre AS empleado,
    NEW.updated_at AS fecha_actualizacion;

-- DELETE con registro de valores eliminados
DELETE FROM productos
WHERE ultima_venta < NOW() - INTERVAL '2 years'
RETURNING
    OLD.id,
    OLD.nombre,
    OLD.categoria,
    'ARCHIVADO_POR_ANTIGUEDAD' AS motivo;

-- INSERT con validación de valores generados
INSERT INTO logs_sistema (evento, usuario, detalles)
VALUES ('LOGIN_FALLIDO', 'admin', '3 intentos fallidos')
RETURNING
    NEW.id,
    NEW.created_at,
    CASE WHEN NEW.severity = 'CRITICAL'
         THEN 'ENVIAR_ALERTA'
         ELSE 'SOLO_REGISTRAR'
    END AS accion;
```

Restricciones Temporales: WITHOUT OVERLAPS

PostgreSQL 18 introduce restricciones `WITHOUT OVERLAPS` para garantizar que no haya solapamiento en rangos temporales .

sql

```
-- Ejemplo: reservas de salas
CREATE TABLE reservas_salas (
    sala_id INT,
```

```
    durante TSRANGE,  -- rango timestamp
    reservado_por TEXT,
    PRIMARY KEY (sala_id, durante WITHOUT OVERLAPS)
);

-- Esto funciona (horarios distintos)
INSERT INTO reservas_salas VALUES (1, '[2026-01-01 09:00, 2026-01-01 10:00)',
'Ana');
INSERT INTO reservas_salas VALUES (1, '[2026-01-01 10:00, 2026-01-01 11:00)',
'Luis');

-- Esto falla (solapamiento)
INSERT INTO reservas_salas VALUES (1, '[2026-01-01 09:30, 2026-01-01 10:30)',
'Carlos');
-- ERROR: conflicting key value violates exclusion constraint

-- Con periodos (range con fechas)
CREATE TABLE contratos (
    empleado_id INT,
    periodo DATERANGE,
    puesto TEXT,
    PRIMARY KEY (empleado_id, periodo WITHOUT OVERLAPS)
);
```

Optimización de consultas con rangos:

sql

```
-- Encontrar solapamientos potenciales (mucho más rápido con la restricción)
SELECT r1.sala_id, r1.durante, r2.durante
FROM reservas_salas r1
JOIN reservas_salas r2 ON r1.sala_id = r2.sala_id
    AND r1.durante && r2.durante  -- operador de solapamiento
    AND r1.ctid < r2.ctid;
```

MERGE Mejorado

La sentencia `MERGE` (UPSERT avanzado) tiene mejor integración con el planificador en PG 18.

sql

```
-- Sincronización de tablas
MERGE INTO inventario AS target
USING inventario_temporal AS source
ON target.producto_id = source.producto_id
WHEN MATCHED AND target.stock != source.stock THEN
    UPDATE SET stock = source.stock, updated_at = NOW()
WHEN MATCHED AND target.precio != source.precio THEN
    UPDATE SET precio = source.precio, updated_at = NOW()
WHEN NOT MATCHED THEN
    INSERT (producto_id, stock, precio, created_at)
    VALUES (source.producto_id, source.stock, source.precio, NOW())
RETURNING
    CASE WHEN OLD IS NULL THEN 'INSERTADO' ELSE 'ACTUALIZADO' END AS operacion,
    COALESCE(OLD.producto_id, NEW.producto_id) AS producto_id;
```

Ejercicio Práctico Integrador (1 hora)

Objetivo: Optimización Completa de una Base de Datos de E-commerce

Escenario

Una tienda online con 5 millones de pedidos está experimentando:

- Consultas lentas en el panel de administración
- Índices fragmentados por uso de UUIDv4
- Estimaciones incorrectas en JOINs multi-tabla

Paso 1: Análisis Inicial

sql

```
-- 1. Identificar tablas con bloat
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS
total_size,
    round(100 * n_dead_tup / nullif(n_live_tup, 0), 2) AS dead_tup_percent
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC
LIMIT 5;

-- 2. Identificar índices no utilizados
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexname::regclass)) AS index_size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
  AND pg_relation_size(indexname::regclass) > 10 * 1024 * 1024  -- > 10MB
ORDER BY pg_relation_size(indexname::regclass) DESC;

-- 3. Verificar estadísticas obsoletas
SELECT
    schemaname,
    tablename,
    last_analyze,
    last_autoanalyze,
    n_mod_since_analyze
FROM pg_stat_user_tables
WHERE n_mod_since_analyze > 10000;
```

Paso 2: Migración a UUIDv7

sql

```
-- Suponiendo tabla pedidos con id UUIDv4 actual
BEGIN;

-- Agregar nueva columna
```

```
ALTER TABLE pedidos ADD COLUMN id_new UUID DEFAULT uuidv7();

-- Poblar en lotes (para evitar bloqueos largos)
DO $$
DECLARE
    batch_size INT := 10000;
    rows_updated INT := 0;
BEGIN
    LOOP
        UPDATE pedidos
        SET id_new = uuidv7()
        WHERE id_new IS NULL
        LIMIT batch_size;

        GET DIAGNOSTICS rows_updated = ROW_COUNT;
        EXIT WHEN rows_updated = 0;

        COMMIT;  -- Commit por lote
    END LOOP;
END $$;

-- Cambiar PK a la nueva columna
ALTER TABLE pedidos DROP CONSTRAINT pedidos_pkey;
ALTER TABLE pedidos ADD PRIMARY KEY (id_new);

-- Recrear FKs referenciando nueva columna
-- (esto varía según las relaciones existentes)

COMMIT;
```

Paso 3: Crear Estadísticas Multivariantes

sql

```
-- Identificar pares de columnas con alta correlación
SELECT
    a.attname AS col1,
    b.attname AS col2,
    pg_stats_correlation(correlation)
FROM pg_stats, ...

-- Crear estadísticas para las combinaciones más relevantes
CREATE STATISTICS pedidos_estado_fecha (dependencies, mcv)
ON estado, fecha_pedido FROM pedidos;

CREATE STATISTICS pedidos_cliente_producto (ndistinct)
ON cliente_id, producto_id FROM pedidos;

-- Actualizar estadísticas
ANALYZE pedidos;
```

Paso 4: Optimizar Consultas Problemáticas

sql

```
-- Consulta lenta identificada: reporte de ventas por cliente
-- Versión original
EXPLAIN (ANALYZE, BUFFERS, TIMING ON)
SELECT
    c.id,
    c.nombre,
    COUNT(p.id) as total_pedidos,
    SUM(p.total) as monto_total
```

```
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha_pedido > NOW() - INTERVAL '6 months'
  AND p.estado = 'COMPLETED'
GROUP BY c.id, c.nombre
ORDER BY monto_total DESC
LIMIT 100;

-- Versión optimizada con Skip Scan y estadísticas
-- Crear índice optimizado para el patrón de consulta
CREATE INDEX idx_pedidos_estado_fecha_incl ON pedidos (estado, fecha_pedido)
INCLUDE (cliente_id, total);

-- La misma consulta ahora usa Skip Scan y el índice cubre todas las columnas
EXPLAIN (ANALYZE, BUFFERS, TIMING ON)
SELECT ... -- misma consulta
```

Paso 5: Implementar Columnas Virtuales

sql

```
-- Agregar columna virtual para precio final (ahorra espacio)
ALTER TABLE pedidos ADD COLUMN
precio_con_envio DECIMAL(12,2) GENERATED ALWAYS AS (
    total +
    CASE WHEN total < 50 THEN 4.99
         WHEN total < 100 THEN 2.99
         ELSE 0
    END
) VIRTUAL;

-- Verificar que no ocupa espacio extra
SELECT pg_size_pretty(pg_total_relation_size('pedidos'));
```

Paso 6: Validación y Monitoreo

sql

```
-- Verificar estadísticas de I/O post-optimización
SELECT * FROM get_top_io_consumers(5);

-- Verificar uso de Skip Scan en consultas
SELECT query, calls, total_time, rows
FROM pg_stat_statements
WHERE query LIKE '%pedidos%'
ORDER BY total_time DESC
LIMIT 10;

-- Crear vista de monitoreo para el equipo
CREATE OR REPLACE VIEW dashboard_optimizacion AS
SELECT
    'Índices no usados' as metric,
    count(*) as value
FROM pg_stat_user_indexes
WHERE idx_scan = 0
UNION ALL
SELECT
    'Dead tuples > 1%',
    count(*)
FROM pg_stat_user_tables
WHERE n_dead_tup * 100 / nullif(n_live_tup, 0) > 1
UNION ALL
SELECT
```

```
    'Estadísticas obsoletas',
    count(*)
FROM pg_stat_user_tables
WHERE n_mod_since_analyze > 10000;
```

Entregables del Ejercicio:

1. Reporte de índices eliminados y creados con justificación
2. Comparativa de tiempo de ejecución antes/después de las optimizaciones
3. Plan de ejecución de la consulta optimizada mostrando Skip Scan
4. Configuración final de `postgresql.conf` para el entorno
5. Script de monitoreo continuo para detectar regresiones

Resumen del Módulo 3

Técnica	Versiones anteriores	PostgreSQL 18
Índices multicolumna	Requieren columna prefijo	Skip Scan automático
UUID como PK	Fragmentación severa	UUIDv7 (monotónico)
Estadísticas	Univariantes	Multivariantes (dependencies, mcv, ndistinct)
Columnas generadas	Solo STORED	VIRTUAL por defecto
NOT NULL constraint	Bloqueo total	NOT VALID + VALIDATE
Migración de versión	Downtime largo	pg_upgrade --swap
RETURNING	Solo NEW	OLD y NEW
Restricciones temporales	Manual con triggers	WITHOUT OVERLAPS nativo

---

Preparación para el Módulo 4

Temas a explorar:

- Implementar OAuth 2.0 nativo para SSO
- Configurar FIPS mode y SSL/TLS 1.3
- Monitoreo de transaction wraparound con nuevas métricas
- Replicación lógica con `pg_stat_subscription_stats`

Lectura recomendada:

- Documentación oficial: "Chapter 68. Multivariate Statistics" (PG 18)

- "UUIDv7 in PostgreSQL 18: Performance Benchmarks"

  ¿Listo para continuar? El Módulo 4 cubrirá seguridad avanzada, autenticación OAuth 2.0, y estrategias de mantenimiento para entornos de alta disponibilidad.

  ok modulo 4 para pg 18

  Leer 7 páginas web

Módulo 4: Seguridad, Mantenimiento y Alta Disponibilidad en PostgreSQL 18 (6 horas)

Objetivos del Módulo

- Implementar autenticación moderna con OAuth 2.0 nativo.
- Configurar cifrado TLS 1.3 y cumplimiento FIPS.
- Dominar la nueva extensión pg_datasentinel para monitoreo avanzado.
- Gestionar transaction wraparound con estimaciones en tiempo real.
- Establecer estrategias de mantenimiento y alta disponibilidad.

4.1. Autenticación Moderna: OAuth 2.0 Nativo (1.5 horas)

Contexto y Motivación

Hasta PostgreSQL 17, la integración con proveedores de identidad (IdP) como Okta, Auth0 o Azure AD requería soluciones externas como PgBouncer con autenticación delegada o proxies. PostgreSQL 18 introduce **autenticación OAuth 2.0 nativa** mediante el mecanismo SASL OAUTHBEARER .

Ventajas clave:

- Tokens de corta duración vs contraseñas permanentes
- Integración directa con SSO corporativo
- Rotación automática de credenciales
- Auditoría centralizada de accesos

Configuración de OAuth 2.0 en pg_hba.conf

El nuevo método `oauth` en `pg_hba.conf` permite configurar autenticación contra cualquier proveedor OAuth 2.0 compatible :

conf

```
# pg_hba.conf - Ejemplo con proveedor OAuth genérico
# Tipo  base de datos  usuario    dirección          método    opciones
hostssl  all           oauth_user 0.0.0.0/0          oauth
client_id=mi_cliente,client_secret=secreto,token_endpoint=https://
auth.provider.com/token

# Ejemplo con Azure AD
hostssl  all           all        0.0.0.0/0          oauth
client_id=azure_client_id,issuer=https://login.microsoftonline.com/tenant_id/
v2.0

# Ejemplo con Okta
hostssl  all           okta_users 0.0.0.0/0          oauth
client_id=okta_client,issuer=https://company.okta.com
```

Parámetros de configuración soportados:

Parámetro	Descripción	Obligatorio
`client_id`	Identificador de la aplicación en el IdP	Sí
`client_secret`	Secreto compartido con el IdP	Sí
`token_endpoint`	URL del endpoint de tokens del IdP	No (se obtiene del issuer)
`issuer`	URL del emisor de tokens (para descubrimiento)	No
`scope`	Scopes solicitados (default: "openid email profile")	No
`timeout`	Timeout para llamadas HTTP al IdP (ms)	No (default: 5000)

Verificación de Autenticación OAuth

sql

```sql
-- Verificar configuración de autenticación actual
SELECT * FROM pg_hba_file_rules WHERE auth_method = 'oauth';

-- Identificar conexiones autenticadas por OAuth (nuevo campo en pg_stat_ssl)
SELECT
    pid,
    usename,
    application_name,
    ssl,
    ssl_identity_mapping  -- NUEVO: mapeo OAuth -> usuario DB
FROM pg_stat_ssl
JOIN pg_stat_activity USING(pid)
WHERE ssl_identity_mapping IS NOT NULL;
```

Mapeo de Identidades OAuth

PostgreSQL 18 permite mapear claims del token JWT a usuarios de base de datos mediante `pg_ident.conf`:

conf

```
# pg_ident.conf
# Nombre_mapa    token_claim_usuari o    usuario_db
```

```
oauth_map        ana.rodriguez@empresa.com    ana_r
oauth_map        carlos.lopez@empresa.com     carlos_l
oauth_map        /^admin\..*@empresa.com$/    admin

# Mapeo por grupos (ej. claim "groups": ["db_analysts"])
oauth_map        analyst                      rol_analyst
```

Migración desde MD5/SCRAM

Advertencia crítica: MD5 está **deprecado en PG 18** y debe ser reemplazado .

sql

```
-- Verificar usuarios que aún usan MD5
SELECT usename, passwd, valuntil
FROM pg_shadow
WHERE passwd LIKE 'md5%';

-- Migrar a SCRAM-SHA-256 (requiere cambiar contraseña)
ALTER USER usuario_con_md5 PASSWORD 'nueva_contraseña_segura';

-- Forzar SCRAM para todas las nuevas contraseñas
ALTER SYSTEM SET password_encryption = 'scram-sha-256';
SELECT pg_reload_conf();
```

4.2. TLS 1.3 y Cifrado Avanzado (1.5 horas)

Nuevo Parámetro: ssl_tls13_ciphers

PostgreSQL 18 introduce control granular sobre los cipher suites de TLS 1.3 mediante el parámetro `ssl_tls13_ciphers` . Este es un cambio significativo porque versiones anteriores solo podían controlar TLS 1.2 y anteriores mediante `ssl_ciphers` .

Evolución de SSL/TLS en PostgreSQL :

Versión	Capacidad SSL/TLS
PG 10-12	Soporte básico, solo cipher control para pre-TLS 1.3
PG 13	Introducción de TLS 1.3 (sin control granular)
PG 15	Mejoras en autenticación por certificado
PG 18	Control granular de cipher suites TLS 1.3

Configuración Hardened para Producción

conf

```
# postgresql.conf - Configuración TLS reforzada
ssl = on
ssl_min_protocol_version = 'TLSv1.2'
ssl_max_protocol_version = 'TLSv1.3'

# Ciphers para TLS 1.2 y anteriores (OPENSSL format)
ssl_ciphers = 'HIGH:!aNULL:!MD5:!3DES'

# Ciphers específicos para TLS 1.3 (NUEVO en PG 18)
```

```
# Por orden de preferencia (separados por :)
ssl_tls13_ciphers = 'TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256'

# Certificados
ssl_cert_file = 'server.crt'
ssl_key_file = 'server.key'
ssl_ca_file = 'ca.crt'
```

Verificación de Conexiones TLS 1.3

sql

```
-- Verificar conexiones TLS activas con detalle de cipher
SELECT
    pid,
    usename,
    ssl,
    ssl_version,
    ssl_cipher,
    ssl_bits,
    ssl_client_dn,
    ssl_identity_mapping
FROM pg_stat_ssl
WHERE ssl = true;

-- Salida esperada:
-- pid | usename | ssl | ssl_version | ssl_cipher                  | ssl_bits
-- 1234| postgres| t   | TLSv1.3     | TLS_AES_256_GCM_SHA384      | 256
```

Verificación desde el cliente :

bash

```
# Usando openssl s_client para verificar TLS 1.3
openssl s_client -starttls postgres -connect localhost:5432 -tls1_3

# Usando psql
psql "host=localhost dbname=postgres sslmode=require"
\conninfo
# SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384)
```

FIPS Mode Validation

PostgreSQL 18 añade la función `fips_mode()` en la extensión `pgcrypto` para verificar si OpenSSL opera en modo FIPS :

sql

```
-- Habilitar pgcrypto si no está disponible
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Verificar estado FIPS
SELECT fips_mode();
-- true = FIPS activo, false = FIPS no activo

-- Nuevo parámetro builtin_crypto_enabled en pgcrypto
-- Valores: on (default), off, fips (deshabilitar cuando FIPS activo)
ALTER SYSTEM SET pgcrypto.builtin_crypto_enabled = 'fips';
SELECT pg_reload_conf();
```

Requisito importante para RHEL 9 con FIPS: Los clientes deben soportar la extensión "Extended Master Secret" (EMS) o usar TLS 1.3 .

4.3. Monitoreo Avanzado con pg_datasentinel (1 hora)

Introducción a pg_datasentinel 1.0

Lanzada en abril de 2026, **pg_datasentinel** es una extensión de observabilidad que aprovecha las nuevas capacidades de PostgreSQL 18 .

Características principales:

- **Monitoreo por backend extendido**: Memoria por proceso, archivos temporales en bytes
- **Visibilidad de contenedores**: CPU quota, memory limits, CPU pressure (Kubernetes/Docker)
- **Estimación de wraparound**: ETA en tiempo real para VACUUM agresivo
- **Captura de logs**: Eventos VACUUM, ANALYZE, checkpoints en buffers anillados

Instalación y Configuración

sql

```
-- Instalación (requiere compilación desde código o paquete)
-- CREATE EXTENSION si está disponible
CREATE EXTENSION IF NOT EXISTS pg_datasentinel;

-- Verificar objetos instalados
\dx pg_datasentinel
```

Monitoreo de Wraparound con Estimaciones en Tiempo Real

Una de las características más valiosas de pg_datasentinel es la **estimación de tiempo para llegar al límite de wraparound** :

sql

```
-- Ver ETA para transaction ID wraparound
SELECT
    datname,
    age(datfrozenxid) AS xid_age,
    -- NUEVO: estimación de tiempo hasta VACUUM agresivo
    pg_datasentinel_eta_vacuum() AS eta_to_aggressive_vacuum,
    pg_datasentinel_eta_wraparound() AS eta_to_wraparound,
    -- Tasa de consumo por hora
    pg_datasentinel_xid_consumption_rate() AS xid_per_hour
FROM pg_database
WHERE datname = current_database();

-- Monitoreo de MXID (multixact) también soportado
SELECT pg_datasentinel_mxid_consumption_rate() AS mxid_per_hour;
```

Monitoreo en Contenedores (Kubernetes/Docker)

pg_datasentinel expone métricas del cgroup del contenedor :

sql

```
-- Ver recursos del contenedor donde corre PostgreSQL
SELECT
    cpu_quota_us,
    cpu_period_us,
```

```
    memory_limit_bytes,
    pg_size_pretty(memory_limit_bytes) AS memory_limit_human,
    memory_usage_bytes,
    pg_size_pretty(memory_usage_bytes) AS memory_usage_human,
    cpu_pressure_percent
FROM pg_datasentinel_container_resources();

-- Verificar si el contenedor tiene límites configurados
-- Útil para debugging de OOM kills
```

Captura y Análisis de Eventos

La extensión registra eventos en buffers anillados accesibles vía SQL :

sql

```
-- Ver últimos eventos de VACUUM
SELECT * FROM pg_datasentinel_vacuum_events
ORDER BY event_time DESC
LIMIT 20;

-- Ver eventos de checkpoint con tiempos detallados
SELECT
    event_time,
    checkpoint_type,
    buffers_written,
    write_time_ms,
    sync_time_ms,
    total_time_ms
FROM pg_datasentinel_checkpoint_events
WHERE event_time > NOW() - INTERVAL '1 hour'
ORDER BY event_time DESC;

-- Ver archivos temporales por sesión
SELECT
    pid,
    usename,
    temp_bytes,
    temp_files,
    pg_datasentinel_temp_file_peak(pid) AS peak_bytes
FROM pg_stat_activity
JOIN pg_datasentinel_backend_stats USING(pid)
WHERE state = 'active';
```

4.4. Mantenimiento y Prevención de Wraparound (1 hora)

Nuevas Métricas de VACUUM en PG 18

Las vistas del sistema ahora incluyen tiempos acumulados de VACUUM :

sql

```
-- Ver tablas con riesgo de wraparound
SELECT
    schemaname,
    tablename,
    n_live_tup,
    n_dead_tup,
    last_vacuum,
```

```
    last_autovacuum,
    vacuum_count,
    autovacuum_count,
    vacuum_time,           -- Tiempo total invertido en VACUUM manual (ms)
    autovacuum_time,       -- Tiempo total invertido en autovacuum (ms)
    age(relfrozenxid) AS xid_age
FROM pg_stat_user_tables
JOIN pg_class ON pg_class.relname = tablename
WHERE age(relfrozenxid) > 100000000  -- Más de 100M transacciones
ORDER BY age(relfrozenxid) DESC;
```

Configuración Avanzada de Autovacuum

sql

```
-- Configuración para tablas con alta tasa de transacciones
ALTER TABLE transacciones SET (
    autovacuum_vacuum_scale_factor = 0.0,        -- Ignorar porcentaje
    autovacuum_vacuum_threshold = 5000,          -- Vacunar cada 5000 cambios
    autovacuum_freeze_min_age = 10000000,        -- Congelar después de 10M tx
    autovacuum_freeze_table_age = 50000000,      -- Forzar vacuum a los 50M tx
    autovacuum_multixact_freeze_min_age = 10000000,
    autovacuum_multixact_freeze_table_age = 50000000
);

-- Tablas críticas pueden tener schedules personalizados
ALTER TABLE logs_auditoria SET (
    autovacuum_vacuum_cost_delay = 0,            -- Sin delay para tablas
críticas
    autovacuum_vacuum_cost_limit = 10000         -- Mayor throughput
);
```

Monitoreo Proactivo con Alertas

sql

```
-- Crear función de alerta para wraparound inminente
CREATE OR REPLACE FUNCTION check_wraparound_risk()
RETURNS TABLE(database text, xid_age bigint, eta_hours numeric) AS $$
BEGIN
    RETURN QUERY
    SELECT
        datname::text,
        age(datfrozenxid),
        -- Estimación simple basada en tasa de consumo (requiere
pg_datasentinel)
        age(datfrozenxid) / NULLIF(
            (SELECT xid_per_hour FROM pg_datasentinel_xid_consumption_rate()
             WHERE datname = current_database()), 0) AS eta_hours
    FROM pg_database
    WHERE age(datfrozenxid) > 1500000000  -- 1.5B tx, wraparound a 2B
    ORDER BY age(datfrozenxid) DESC;
END;
$$ LANGUAGE plpgsql;

-- Uso: SELECT * FROM check_wraparound_risk();
```

4.5. Wire Protocol 3.2 y Cancelación Segura (0.5 horas)

Novedades del Protocolo 3.2

PostgreSQL 18 introduce el primer cambio al protocolo wire en más de 20 años (desde PG 7.4 en 2003) .

Mejora crítica: Cancel request keys de **256 bits** (vs 32 bits anteriormente).

Aspecto	PG 17 y anteriores	PG 18 con protocolo 3.2
Longitud cancel key	32 bits	256 bits
Espacio de búsqueda	~4 mil millones	~2^256 (prácticamente infinito)
Vulnerabilidad a brute-force	Posible	Imposible
Compatibilidad hacia atrás	N/A	Full (usa 32 bits con clientes antiguos)

Verificación del Protocolo en Uso

sql

```
-- PostgreSQL 18 no expone directamente el protocolo versión via SQL
-- Se puede verificar mediante libpq en el cliente
```

c

```
// Código cliente para verificar protocol version (usando libpq)
int protocol_version = PQfullProtocolVersion(conn);
if (protocol_version >= 30002) {  // 3.2 = 30002
    printf("Usando protocolo 3.2 con cancel keys de 256 bits\n");
}
```

Para la mayoría de las aplicaciones: La actualización es transparente. Los controladores de base de datos deberán actualizarse para optar al protocolo 3.2 .

4.6. Integridad de Datos: Page Checksums por Defecto (0.5 horas)

Checksums Habilitados por Default

PostgreSQL 18 activa **data checksums por defecto** en `initdb` . Esto es un cambio significativo respecto a versiones anteriores donde era opcional.

Qué hacen los checksums: Cada página de 8KB tiene un hash que verifica su integridad. Al leer, PostgreSQL valida que la página no haya sido corrompida por errores de hardware o del sistema de archivos.

sql

```
-- Verificar si checksums están activos
SHOW data_checksums;
-- En PG 18 con cluster nuevo: on
```

```
-- Verificar integridad de una tabla específica
SELECT pg_checksum_relation('mi_tabla'::regclass);

-- Ver estadísticas de checksums (nuevo en PG 18)
SELECT
    datname,
    checksum_failures,
    checksum_last_failure
FROM pg_stat_database
WHERE checksum_failures > 0;
```

Consideraciones para pg_upgrade: Si el cluster origen no tenía checksums, el cluster destino tampoco los tendrá a menos que se use `pg_upgrade --no-clone` con `pg_checksums --enable` antes .

Ejercicio Práctico Integrador (1 hora)

Objetivo: Implementar Seguridad y Monitoreo de Extremo a Extremo

Paso 1: Configurar TLS 1.3 Hardened

bash

```
# Generar certificados autofirmados para pruebas
openssl req -new -x509 -days 365 -nodes -text \
    -out server.crt -keyout server.key \
    -subj "/CN=postgres.local"

chmod 600 server.key
chown postgres:postgres server.crt server.key
mv server.crt server.key /var/lib/pgsql/data/
```

sql

```
-- Configurar PostgreSQL
ALTER SYSTEM SET ssl = 'on';
ALTER SYSTEM SET ssl_min_protocol_version = 'TLSv1.2';
ALTER SYSTEM SET ssl_max_protocol_version = 'TLSv1.3';
ALTER SYSTEM SET ssl_tls13_ciphers = 'TLS_AES_256_GCM_SHA384';
ALTER SYSTEM SET ssl_ciphers = 'HIGH:!aNULL:!MD5';
SELECT pg_reload_conf();
```

Paso 2: Migrar Autenticación a SCRAM

sql

```
-- Verificar usuarios con MD5
SELECT usename FROM pg_shadow WHERE passwd LIKE 'md5%';

-- Cambiar contraseña a cada uno (reemplazar 'usuario')
ALTER USER usuario PASSWORD 'nueva_contraseña_segura';

-- Verificar que ahora usan SCRAM
SELECT usename,
       CASE WHEN passwd LIKE 'SCRAM-SHA-256%' THEN 'SCRAM'
            WHEN passwd LIKE 'md5%' THEN 'MD5 (inseguro)'
            ELSE 'otro' END AS auth_method
FROM pg_shadow;
```

Paso 3: Instalar y Configurar pg_datasentinel

bash

```bash
# Desde el código fuente (asumiendo PostgreSQL 18 instalado)
git clone https://github.com/datasentinel/pg_datasentinel
cd pg_datasentinel
make USE_PGXS=1
make install USE_PGXS=1
```

sql

```sql
-- En PostgreSQL
CREATE EXTENSION pg_datasentinel;

-- Verificar estado de wraparound
SELECT datname, age(datfrozenxid) as xid_age,
       pg_size_pretty(pg_database_size(datname)) as size
FROM pg_database
ORDER BY age(datfrozenxid) DESC;

-- Si pg_datasentinel tiene la función ETA
SELECT pg_datasentinel_eta_wraparound() AS horas_restantes;
```

Paso 4: Configurar pg_hba para Seguridad

conf

```conf
# pg_hba.conf - Configuración final
# TYPE  DATABASE    USER        ADDRESS         METHOD
# Conexiones locales con SCRAM (seguro)
hostssl  all         all         127.0.0.1/32    scram-sha-256
hostssl  all         all         ::1/128         scram-sha-256

# Conexiones de aplicación (ejemplo con OAuth)
hostssl  app_db      app_user    10.0.0.0/8      oauth
client_id=app_client,token_endpoint=https://auth.internal/token

# Rechazar explícitamente MD5 y conexiones no SSL
hostnossl all        all         0.0.0.0/0       reject
host      all        all         0.0.0.0/0       reject
```

sql

```sql
-- Recargar configuración sin reiniciar
SELECT pg_reload_conf();

-- Verificar reglas activas
SELECT * FROM pg_hba_file_rules WHERE error IS NULL;
```

Paso 5: Monitoreo y Alertas

sql

```sql
-- Crear vista de monitoreo de seguridad
CREATE OR REPLACE VIEW security_dashboard AS
SELECT
    'Conexiones SSL activas' AS metric,
    COUNT(*) AS value
FROM pg_stat_ssl WHERE ssl = true
UNION ALL
SELECT
    'Conexiones TLS 1.3',
    COUNT(*)
FROM pg_stat_ssl WHERE ssl_version = 'TLSv1.3'
```

```
UNION ALL
SELECT
    'Usuarios con MD5',
    COUNT(*)
FROM pg_shadow WHERE passwd LIKE 'md5%'
UNION ALL
SELECT
    'Checksums activos',
    CASE WHEN current_setting('data_checksums') = 'on' THEN 1 ELSE 0 END::text;

-- Verificar periódicamente
SELECT * FROM security_dashboard;
```

Resumen del Módulo 4

Componente	Versiones anteriores	PostgreSQL 18
Autenticación	MD5/SCRAM	OAuth 2.0 nativo
TLS control	Solo pre-TLS 1.3	Cifrados TLS 1.3 granulares
FIPS compliance	Manual	`fips_mode()` + `builtin_crypto_enabled`
Cancel keys	32 bits (vulnerable)	256 bits (seguro)
Wraparound monitoreo	Solo edad actual	ETA y tasa de consumo
Checksums	Opcional	Activado por defecto
Monitoreo contenedores	Limitado	cgroup metrics nativas

Preparación para el Módulo 5

Temas a explorar:

- Replicación lógica mejorada con `pg_stat_subscription_stats`
- Particionamiento avanzado y foreign keys
- JSONB optimizaciones
- Full-Text Search con nuevas collations

Lectura recomendada:

- Documentación oficial: "Chapter 20. Client Authentication" (PG 18)
- pg_datasentinel GitHub repository

¿Listo para continuar? El Módulo 5 cubrirá SQL avanzado con las nuevas cláusulas (`RETURNING OLD/NEW`, `WITHOUT OVERLAPS`), optimización de JSONB y Full-Text Search.

desarrolla el modulo 5 para pg18

Leer 10 páginas web

Módulo 5: SQL Avanzado y Optimización de Datos Semiestructurados en PostgreSQL 18 (6 horas)

Objetivos del Módulo

- Dominar las nuevas cláusulas SQL para auditoría y transformación de datos.
- Implementar restricciones temporales nativas con `WITHOUT OVERLAPS`.
- Optimizar el almacenamiento y consulta de JSONB con las mejoras de PG 18.
- Utilizar la nueva collation `PG_UNICODE_FAST` para búsquedas de texto más rápidas.
- Aplicar técnicas avanzadas de modelado con tipos de datos mejorados.

5.1. RETURNING con OLD/NEW: Auditoría y Transformación (1.5 horas)

Contexto y Motivación

Desde PostgreSQL 8.2, la cláusula `RETURNING` permitía devolver valores después de una operación DML, pero solo los **nuevos valores** (o los eliminados en DELETE). Para obtener los valores anteriores a una actualización, era necesario usar trucos poco intuitivos como CTEs con `FOR UPDATE` o verificar `xmax = 0` en combinación con `ON CONFLICT`.

Ejemplo del problema antes de PG 18:

sql

```
-- Obtener valores antes y después de un UPDATE requería CTE complejos
WITH x AS (SELECT id, v FROM t WHERE id IN (1,2,3) FOR UPDATE),
     u AS (UPDATE t SET v = 123 FROM x WHERE t.id = x.id RETURNING t.id, t.v)
SELECT COALESCE(x.id, u.id), x.v AS before, u.v AS after
FROM x FULL OUTER JOIN u USING (id);
```

PostgreSQL 18 resuelve este problema permitiendo acceso directo a `OLD` y `NEW` en la cláusula `RETURNING` para `INSERT`, `UPDATE`, `DELETE` y `MERGE`.

Sintaxis y Casos de Uso

Sintaxis Básica

sql

```
-- Acceso directo a columnas individuales
RETURNING OLD.columna, NEW.columna

-- Todas las columnas con OLD.* y NEW.*
RETURNING OLD.*, NEW.*
```

```
-- Con alias personalizados para evitar conflictos
RETURNING WITH (OLD AS o, NEW AS n) o.columna, n.columna
```

La sintaxis con `WITH (OLD AS alias, NEW AS alias)` es útil cuando los nombres "old" y "new" ya están definidos en el contexto, como dentro de funciones trigger .

UPDATE: Comparación de Valores

sql

```
-- Actualización simple con comparación de cambios
UPDATE productos
SET precio = precio * 1.10,
    updated_at = NOW()
WHERE categoria = 'ELECTRONICA'
RETURNING
    OLD.precio AS precio_anterior,
    NEW.precio AS precio_nuevo,
    OLD.nombre AS producto,
    ROUND((NEW.precio - OLD.precio) / OLD.precio * 100, 2) AS
incremento_porcentual,
    NEW.updated_at AS fecha_actualizacion;
```

Salida esperada:

text

```
 precio_anterior | precio_nuevo | producto     | incremento_porcentual |
fecha_actualizacion
-----------------|--------------|--------------|----------------------|---------
------------
 100.00          | 110.00       | Laptop Pro   | 10.00                | 2026-04-
12 10:30:00
 50.00           | 55.00        | Mouse Wireless| 10.00                | 2026-
04-12 10:30:00
```

DELETE: Registro de Auditoría

sql

```
-- Eliminar productos obsoletos con registro completo
DELETE FROM productos
WHERE ultima_venta < NOW() - INTERVAL '2 years'
RETURNING
    OLD.id,
    OLD.nombre,
    OLD.categoria,
    OLD.precio,
    OLD.ultima_venta,
    'ARCHIVADO_POR_ANTIGUEDAD' AS motivo,
    CURRENT_USER AS eliminado_por;
```

INSERT: Validación de Valores Generados

sql

```
-- Insertar con verificación de valores generados automáticamente
INSERT INTO logs_sistema (evento, usuario, severidad, detalles)
VALUES
    ('LOGIN_FALLIDO', 'admin', 'MEDIUM', '3 intentos fallidos en 5 minutos'),
    ('BACKUP_COMPLETADO', 'sistema', 'INFO', 'Backup diario exitoso')
RETURNING
    NEW.id,
```

```
    NEW.created_at,
    CASE
        WHEN NEW.severidad = 'CRITICAL' THEN 'ENVIAR_ALERTA_SLACK'
        WHEN NEW.severidad = 'HIGH' THEN 'ENVIAR_EMAIL_A_ADMIN'
        ELSE 'SOLO_REGISTRAR'
    END AS accion_requerida;
```

Caso Práctico: UPSERT con Detección de Nuevas Filas

Una de las aplicaciones más útiles es diferenciar entre filas insertadas y actualizadas en una operación `ON CONFLICT`.

Antes de PG 18 (usando truco de xmax):

sql

```
INSERT INTO webhook (id, data)
VALUES (@id, @data)
ON CONFLICT (id) DO UPDATE SET id = webhook.id  -- Force upsert to return a row
RETURNING webhook.*, (xmax = 0) AS is_new;  -- Truco poco intuitivo
```

En PostgreSQL 18 (legible y confiable):

sql

```
INSERT INTO webhook (id, data, created_at, updated_at)
VALUES (@id, @data, NOW(), NOW())
ON CONFLICT (id) DO UPDATE
SET data = EXCLUDED.data,
    updated_at = NOW()
RETURNING
    webhook.*,
    (OLD IS NULL)::boolean AS is_new;  -- Simple y claro
```

Ejercicio Práctico 5.1.1: Sistema de Auditoría Completo

sql

```
-- Crear tabla de auditoría
CREATE TABLE auditoria_productos (
    id SERIAL PRIMARY KEY,
    operacion TEXT NOT NULL,
    producto_id INT NOT NULL,
    nombre_anterior TEXT,
    nombre_nuevo TEXT,
    precio_anterior DECIMAL(10,2),
    precio_nuevo DECIMAL(10,2),
    usuario TEXT NOT NULL,
    fecha_cambio TIMESTAMPTZ DEFAULT NOW()
);

-- Función de auditoría usando RETURNING con OLD/NEW
CREATE OR REPLACE FUNCTION registrar_auditoria()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        INSERT INTO auditoria_productos (operacion, producto_id,
nombre_anterior, nombre_nuevo,
                                         precio_anterior, precio_nuevo,
usuario)
        SELECT 'UPDATE', NEW.id, OLD.nombre, NEW.nombre, OLD.precio, NEW.precio,
CURRENT_USER
        FROM (SELECT OLD.*, NEW.*) AS datos;
```

```
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO auditoria_productos (operacion, producto_id,
nombre_anterior, precio_anterior, usuario)
        VALUES ('DELETE', OLD.id, OLD.nombre, OLD.precio, CURRENT_USER);
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Trigger que usa la auditoría
CREATE TRIGGER trg_auditoria_productos
AFTER UPDATE OR DELETE ON productos
FOR EACH ROW EXECUTE FUNCTION registrar_auditoria();
```

5.2. Restricciones Temporales con WITHOUT OVERLAPS (1.5 horas)

Fundamentos de Rangos en PostgreSQL

Antes de usar `WITHOUT OVERLAPS`, es esencial entender los tipos de rango que PostgreSQL proporciona :

Tipo de Rango	Descripción	Ejemplo de uso
`daterange`	Rango de fechas (sin hora)	Contratos, licencias
`tsrange`	Rango de timestamp (sin zona horaria)	Eventos locales
`tstzrange`	Rango con zona horaria	Reservas internacionales
`int4range`	Rango de enteros	Versiones, niveles

Notación de rangos:

- `[` = límite inferior inclusivo
- `(` = límite inferior exclusivo
- `]` = límite superior inclusivo
- `)` = límite superior exclusivo

El formato `[)` es el más común para períodos temporales porque crea rangos contiguos sin solapamiento .

Sintaxis WITHOUT OVERLAPS

PostgreSQL 18 introduce `WITHOUT OVERLAPS` como una forma más legible de crear exclusion constraints para rangos .

Comparación: Antes vs. Después

sql

```
-- Antes de PG 18 (requería extensión btree_gist)
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

```
CREATE TABLE reservas_old (
    sala_id INTEGER,
    periodo TSTZRANGE,
    EXCLUDE USING gist (sala_id WITH =, periodo WITH &&)
);

-- PostgreSQL 18 (sintaxis limpia y nativa)
CREATE TABLE reservas (
    sala_id INTEGER,
    periodo TSTZRANGE NOT NULL,
    cliente TEXT NOT NULL,
    PRIMARY KEY (sala_id, periodo WITHOUT OVERLAPS)
);
```

Caso 1: Sistema de Reservas de Salas

sql

```
-- Crear tabla con restricción temporal
CREATE TABLE reservas_salas (
    sala_id INTEGER NOT NULL,
    periodo TSTZRANGE NOT NULL,
    cliente_nombre TEXT NOT NULL,
    proposito TEXT,
    PRIMARY KEY (sala_id, periodo WITHOUT OVERLAPS)
);

-- Insertar reservas válidas (sin solapamiento)
INSERT INTO reservas_salas (sala_id, periodo, cliente_nombre, proposito) VALUES
    (101, '[2026-04-15 09:00, 2026-04-15 11:00)', 'Ana Rodríguez', 'Reunión
equipo'),
    (101, '[2026-04-15 11:00, 2026-04-15 13:00)', 'Carlos López', 'Entrevista'),
    (102, '[2026-04-15 10:00, 2026-04-15 12:00)', 'María García',
'Presentación');

-- Intentar insertar reserva solapada (será rechazada)
INSERT INTO reservas_salas (sala_id, periodo, cliente_nombre, proposito)
VALUES (101, '[2026-04-15 10:30, 2026-04-15 12:00)', 'Juan Pérez',
'Sobrevuelo');

-- ERROR: conflicting key value violates exclusion constraint
"reservas_salas_pkey"
```

Caso 2: Contratos de Empleados sin Solapamiento

sql

```
-- Empleados no pueden tener dos contratos activos simultáneamente
CREATE TABLE contratos_empleados (
    empleado_id INTEGER NOT NULL,
    periodo DATERANGE NOT NULL,
    puesto TEXT NOT NULL,
    salario DECIMAL(10,2) NOT NULL,
    UNIQUE (empleado_id, periodo WITHOUT OVERLAPS)
);

-- Insertar contratos válidos
INSERT INTO contratos_empleados (empleado_id, periodo, puesto, salario) VALUES
    (1001, '[2025-01-01, 2025-12-31)', 'Desarrollador Senior', 75000),
    (1001, '[2026-01-01, 2026-12-31)', 'Tech Lead', 90000),
    (1002, '[2025-06-01, 2025-12-31)', 'QA Engineer', 55000);
```

```
-- Query para encontrar período actual de un empleado
SELECT * FROM contratos_empleados
WHERE empleado_id = 1001
  AND periodo @> CURRENT_DATE;
```

WITHOUT OVERLAPS con UNIQUE vs PRIMARY KEY

Puedes usar `WITHOUT OVERLAPS` tanto en `PRIMARY KEY` como en `UNIQUE` constraints :

sql

```
-- Usando UNIQUE (cuando hay una PK separada)
CREATE TABLE asignaciones_proyectos (
    id SERIAL PRIMARY KEY,
    empleado_id INTEGER NOT NULL,
    proyecto_id INTEGER NOT NULL,
    periodo DATERANGE NOT NULL,
    UNIQUE (empleado_id, periodo WITHOUT OVERLAPS)
);
```

Consideraciones Técnicas

1. **Índices GiST automáticos**: PostgreSQL crea automáticamente un índice GiST para la constraint, no un B-tree .
2. **Extensión btree_gist**: Si la constraint incluye columnas que no son de tipo rango (como `sala_id`), se necesita la extensión `btree_gist` .
3. **Operador de solapamiento**: Internamente, `WITHOUT OVERLAPS` usa el operador && para rangos .

Ejercicio Práctico 5.2.1: Sistema de Turnos Médicos

sql

```
-- Crear tabla de citas médicas con restricción temporal
CREATE TABLE citas_medicas (
    id SERIAL PRIMARY KEY,
    medico_id INTEGER NOT NULL,
    paciente_id INTEGER NOT NULL,
    horario TSTZRANGE NOT NULL,
    motivo TEXT,
    UNIQUE (medico_id, horario WITHOUT OVERLAPS),  -- Un médico no puede tener
citas solapadas
    UNIQUE (paciente_id, horario WITHOUT OVERLAPS) -- Un paciente no puede tener
citas solapadas
);

-- Insertar citas válidas
INSERT INTO citas_medicas (medico_id, paciente_id, horario, motivo) VALUES
    (1, 100, '[2026-04-20 09:00, 2026-04-20 09:30)', 'Control anual'),
    (1, 101, '[2026-04-20 09:30, 2026-04-20 10:00)', 'Dolor de cabeza'),
    (2, 100, '[2026-04-20 11:00, 2026-04-20 11:30)', 'Seguimiento');  -- Mismo
paciente, diferente médico, OK

-- Consultar disponibilidad del médico
SELECT medico_id, horario
FROM citas_medicas
WHERE medico_id = 1
```

```
  AND horario && tstzrange('2026-04-20 09:15', '2026-04-20 09:45');
```

5.3. JSONB Avanzado en PostgreSQL 18 (1.5 horas)

JSON vs JSONB: ¿Cuál Usar?

PostgreSQL 18 mantiene la recomendación de usar **JSONB** como default para casi todos los casos :

Característica	JSON	JSONB
Almacenamiento	Texto original	Binario optimizado
Velocidad escritura	Más rápida	Ligeramente más lenta
Velocidad lectura	Más lenta (re-parsea)	Mucho más rápida
Indexación	No soportada	GIN, expresión BTREE
Preservación formato	Sí (whitespace, orden)	No
Recomendación PG 18	Solo para compatibilidad	**Default**

Mejoras de Rendimiento en PG 18

PostgreSQL 18 introduce :

- **Operadores JSON optimizados** para consultas frecuentes
- Mejor integración con el nuevo subsistema **Async I/O**
- **GIN indexes paralelos** para construcción más rápida

Operadores JSONB Esenciales

sql

```
-- Operador de contención (@>): ¿El documento contiene este subdocumento?
SELECT * FROM productos
WHERE datos @> '{"categoria": "electrónica", "en_stock": true}';

-- Operador de existencia (?) : ¿Existe esta clave?
SELECT * FROM productos
WHERE datos ? 'descuento';

-- Operador de acceso a elementos (->, ->>)
SELECT
    datos->>'nombre' AS nombre,
    datos->'especificaciones'->>'procesador' AS procesador
FROM productos;

-- Operador de concatenación (||) para combinar JSONB
UPDATE productos
SET datos = datos || '{"actualizado": true, "version": 2}'
WHERE id = 1;
```

Indexación JSONB en PG 18

sql

```
-- Índice GIN completo (para consultas de contención)
CREATE INDEX idx_productos_datos_gin ON productos USING gin (datos);

-- Índice GIN con jsonb_path_ops (más rápido y compacto para @>)
CREATE INDEX idx_productos_path_ops ON productos USING gin (datos
jsonb_path_ops);

-- Índice de expresión BTREE (para un campo específico)
CREATE INDEX idx_productos_precio ON productos ((datos->>'precio'));

-- Índice parcial para documentos con cierta clave
CREATE INDEX idx_productos_oferta ON productos ((datos->>'descuento'))
WHERE datos ? 'descuento';
```

JSONPath: Consultas Avanzadas

PostgreSQL soporta el lenguaje **JSONPath** para consultas complejas dentro de documentos :

sql

```
-- Encontrar productos con precio entre 100 y 500
SELECT * FROM productos
WHERE datos @? '$.precio ? (@ >= 100 && @ <= 500)';

-- Productos con reviews de al menos 4 estrellas
SELECT * FROM productos
WHERE datos @? '$.reviews[*].rating ? (@ >= 4)';

-- Extraer todos los tags de productos electrónicos
SELECT jsonb_path_query(datos, '$.tags[*]') AS tag
FROM productos
WHERE datos->>'categoria' = 'electrónica';
```

Funciones JSONB Útiles

sql

```
-- jsonb_set: Actualizar un valor anidado
UPDATE productos
SET datos = jsonb_set(
    datos,
    '{especificaciones,memoria}',
    '"16GB"'
    -- , true  → crear si no existe
)
WHERE datos->>'nombre' = 'Laptop Pro';

-- jsonb_insert: Insertar en un array
UPDATE productos
SET datos = jsonb_insert(
    datos,
    '{tags, 0}',  -- Posición 0 del array tags
    '"nuevo"'
)
WHERE id = 1;

-- jsonb_strip_nulls: Eliminar claves con valor null
UPDATE productos
SET datos = jsonb_strip_nulls(datos)
WHERE datos ? 'campo_temporal';

-- jsonb_typeof: Obtener tipo de un valor
```

```
SELECT
    datos->>'nombre',
    jsonb_typeof(datos->'precio') AS tipo_precio
FROM productos;
```

Caso Práctico: Tienda E-commerce con JSONB

sql

```
-- Esquema híbrido: columnas relacionales + JSONB para atributos variables
CREATE TABLE productos (
    id SERIAL PRIMARY KEY,
    sku TEXT UNIQUE NOT NULL,
    nombre TEXT NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    categoria TEXT NOT NULL,
    -- Atributos variables del producto
    atributos JSONB DEFAULT '{}'::jsonb,
    -- Metadatos del producto
    metadata JSONB DEFAULT '{"creado_en": null, "modificado_en": null, "tags":
[]}'::jsonb,
    -- Índice GIN para búsqueda en atributos
    CHECK (jsonb_typeof(atributos) = 'object')
);

-- Índices optimizados
CREATE INDEX idx_productos_atributos_gin ON productos USING gin (atributos);
CREATE INDEX idx_productos_precio ON productos (precio);
CREATE INDEX idx_productos_categoria ON productos (categoria);

-- Insertar productos con diferentes atributos
INSERT INTO productos (sku, nombre, precio, categoria, atributos) VALUES
    ('LP-001', 'Laptop Gamer X1', 1299.99, 'electrónica',
     '{"procesador": "Intel i7", "ram": "16GB", "almacenamiento": "512GB SSD",
"gpu": "RTX 4060"}'::jsonb),
    ('MS-002', 'Mouse Inalámbrico', 49.99, 'accesorios',
     '{"conexion": "USB-C", "dpi": 3200, "bateria": "40h"}'::jsonb),
    ('MN-003', 'Monitor 4K', 399.99, 'electrónica',
     '{"resolucion": "3840x2160", "tamaño": "27 pulgadas", "hz": 144}'::jsonb);

-- Consulta: Productos con ciertas especificaciones
SELECT nombre, precio, atributos->>'ram' AS ram
FROM productos
WHERE atributos @> '{"procesador": "Intel i7"}'
  AND atributos ? 'gpu'
ORDER BY precio DESC;

-- Actualizar atributos específicos
UPDATE productos
SET atributos = jsonb_set(atributos, '{garantia}', '"2 años"')
WHERE categoria = 'electrónica';
```

5.4. PG_UNICODE_FAST: Colación Rápida para Texto (1 hora)

¿Qué es PG_UNICODE_FAST?

PostgreSQL 18 introduce una nueva colación llamada `PG_UNICODE_FAST` que proporciona :

1. **Semántica Unicode completa** para transformaciones de mayúsculas/minúsculas (ej. 'ß' → 'SS')
2. **Alta velocidad** usando `memcmp` para comparaciones
3. Soporte para la nueva función `casefold()` para comparaciones case-insensitive

Contexto técnico :

- `PG_C_UTF8`: Semántica Unicode "simple" (más rápida pero menos precisa)
- `PG_UNICODE_FAST`: Semántica Unicode "completa" + velocidad optimizada

Configuración y Uso

sql

```
-- Ver collations disponibles
SELECT collname, collprovider, collversion
FROM pg_collation
WHERE collname LIKE 'PG%';

-- Crear tabla usando PG_UNICODE_FAST
CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    nombre TEXT COLLATE "PG_UNICODE_FAST",
    email TEXT
);

-- Ordenamiento correcto con caracteres Unicode
INSERT INTO clientes (nombre) VALUES
    ('straße'),  -- alemán: calle
    ('Strasse'), -- variante alternativa
    ('Zürich'),
    ('Zurich');

SELECT nombre FROM clientes ORDER BY nombre;
-- 'straße' y 'Strasse' se ordenan juntos correctamente
```

Función casefold() para Comparaciones Case-Insensitive

La nueva función `casefold()` permite comparaciones sin distinción de mayúsculas/minúsculas con semántica Unicode completa :

sql

```
-- Búsqueda case-insensitive con casefold()
SELECT * FROM clientes
WHERE casefold(nombre) = casefold('STRASSE');
-- Encuentra tanto 'straße' como 'Strasse'

-- Índice para búsquedas case-insensitive rápidas
CREATE INDEX idx_clientes_nombre_casefold ON clientes (casefold(nombre));

-- Ahora la búsqueda usará el índice
SELECT * FROM clientes
WHERE casefold(nombre) = casefold('ZURICH');
```

Comparación de Rendimiento

sql

```
-- Prueba de rendimiento: búsqueda con LIKE vs casefold
\timing on

-- Sin índice (búsqueda secuencial)
SELECT COUNT(*) FROM clientes WHERE casefold(nombre) = casefold('zurich');
-- Tiempo: ~50ms para 1M registros

-- Con índice (búsqueda por índice)
SELECT COUNT(*) FROM clientes WHERE casefold(nombre) = casefold('zurich');
-- Tiempo: ~1ms
```

LIKE con Collations No Determinísticas

PG 18 permite usar `LIKE` con collations no determinísticas :

sql

```
-- Búsqueda case-insensitive con LIKE
SELECT * FROM clientes
WHERE nombre COLLATE "PG_UNICODE_FAST" LIKE '%str%';
-- Encuentra 'straße', 'Strasse', 'Straße'
```

Advertencia para pg_upgrade

Si actualizas desde una versión anterior, ten en cuenta que los índices de **full text search** y **pg_trgm** deben ser reindexados después de `pg_upgrade` porque ahora usan el collation provider por defecto del cluster .

5.5. Nuevos Tipos y Funciones de Fecha/Hora (0.5 horas)

UUIDv7: Generación Monotónica

UUIDv7 es una de las adiciones más importantes en PG 18 para rendimiento de índices :

sql

```
-- Generar UUIDv7 (timestamp-ordered)
SELECT uuidv7();
-- Resultado: 01980de8-ad3d-715c-b739-faf2bb1a7aad

-- Extraer timestamp del UUID
SELECT uuid_extract_timestamp(uuidv7());
-- Resultado: 2026-04-12 10:30:45.123+00

-- Verificar versión del UUID
SELECT uuid_extract_version(uuidv7());  -- Retorna 7

-- UUIDv4 como alias de gen_random_uuid()
SELECT uuidv4();  -- Equivalente a gen_random_uuid()
```

Benchmark: UUIDv4 vs UUIDv7

sql

```
-- Crear tablas de prueba
CREATE TABLE test_uuid_v4 (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), data
TEXT);
CREATE TABLE test_uuid_v7 (id UUID PRIMARY KEY DEFAULT uuidv7(), data TEXT);
```

```
-- Insertar 1M registros en cada una
INSERT INTO test_uuid_v4 (data) SELECT md5(i::text) FROM generate_series(1,
1000000) i;
INSERT INTO test_uuid_v7 (data) SELECT md5(i::text) FROM generate_series(1,
1000000) i;

-- Comparar tamaño del índice
SELECT
    tablename,
    pg_size_pretty(pg_indexes_size(tablename::regclass)) AS index_size
FROM (VALUES ('test_uuid_v4'), ('test_uuid_v7')) AS t(tablename);

-- Resultado esperado: UUIDv7 tiene índice ~30% más pequeño
```

Columnas Generadas Virtuales (Default)

PG 18 hace que **VIRTUAL** sea el valor predeterminado para columnas generadas :

sql

```
-- Virtual (default) - no ocupa espacio en disco
CREATE TABLE productos (
    id SERIAL PRIMARY KEY,
    precio_base DECIMAL(10,2),
    iva DECIMAL(4,2) DEFAULT 21.00,
    precio_final DECIMAL(10,2) GENERATED ALWAYS AS (
        precio_base * (1 + iva/100)
    ) VIRTUAL  -- VIRTUAL es opcional, es el default
);

-- Stored - ocupa espacio pero se puede indexar
CREATE TABLE productos_con_almacenamiento (
    id SERIAL PRIMARY KEY,
    precio_base DECIMAL(10,2),
    iva DECIMAL(4,2) DEFAULT 21.00,
    precio_con_iva DECIMAL(10,2) GENERATED ALWAYS AS (
        precio_base * (1 + iva/100)
    ) STORED  -- Se almacena físicamente
);

-- Indexar columna STORED
CREATE INDEX idx_precio_iva ON productos_con_almacenamiento (precio_con_iva);
```

Nuevas Funciones de Texto

sql

```
-- casefold() para normalización case-insensitive
SELECT casefold('Straße');   -- 'strasse'
SELECT casefold('ß');        -- 'ss'

-- upper()/lower() mejorados con PG_UNICODE_FAST
SELECT upper('straße' COLLATE "PG_UNICODE_FAST");  -- 'STRASSE'
SELECT lower('STRASSE' COLLATE "PG_UNICODE_FAST"); -- 'strasse'
```

Ejercicio Práctico Integrador (1 hora)

Objetivo: Construir un Sistema de Gestión de Eventos con todas las características de PG 18

Escenario

Una plataforma de gestión de eventos necesita:

- Reservas de salas sin solapamiento
- Auditoría completa de cambios
- Almacenamiento flexible de metadatos de eventos
- Búsqueda case-insensitive en títulos

Paso 1: Crear el Esquema Principal

sql

```
-- Tabla de eventos con JSONB para atributos flexibles
CREATE TABLE eventos (
    id UUID PRIMARY KEY DEFAULT uuidv7(),  -- UUIDv7 para mejor rendimiento de
índices
    titulo TEXT NOT NULL COLLATE "PG_UNICODE_FAST",
    descripcion TEXT,
    categoria TEXT NOT NULL,
    precio_base DECIMAL(10,2) NOT NULL,
    -- Atributos variables por tipo de evento
    atributos JSONB DEFAULT '{}'::jsonb,
    -- Metadatos del evento
    metadata JSONB DEFAULT jsonb_build_object(
        'creado_en', NOW(),
        'creado_por', CURRENT_USER,
        'modificado_en', null,
        'tags', []'::jsonb
    ),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Índices optimizados
CREATE INDEX idx_eventos_titulo_casefold ON eventos (casefold(titulo));
CREATE INDEX idx_eventos_atributos_gin ON eventos USING gin (atributos);
CREATE INDEX idx_eventos_created_at ON eventos (created_at);
```

Paso 2: Tabla de Reservas con WITHOUT OVERLAPS

sql

```
-- Tabla de reservas con restricción temporal
CREATE TABLE reservas_eventos (
    id SERIAL PRIMARY KEY,
    evento_id UUID NOT NULL REFERENCES eventos(id) ON DELETE CASCADE,
    sala_id INTEGER NOT NULL,
    horario TSTZRANGE NOT NULL,
    capacidad_maxima INTEGER NOT NULL,
    -- Restricción: misma sala no puede tener eventos solapados
    UNIQUE (sala_id, horario WITHOUT OVERLAPS)
```

```
);
```

Paso 3: Sistema de Auditoría con OLD/NEW

sql

```
-- Tabla de auditoría
CREATE TABLE auditoria_eventos (
    id SERIAL PRIMARY KEY,
    evento_id UUID NOT NULL,
    operacion TEXT NOT NULL,
    cambios JSONB NOT NULL,
    usuario TEXT NOT NULL,
    fecha TIMESTAMPTZ DEFAULT NOW()
);

-- Función de trigger usando OLD/NEW
CREATE OR REPLACE FUNCTION audit_eventos()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        INSERT INTO auditoria_eventos (evento_id, operacion, cambios, usuario)
        SELECT
            NEW.id,
            'UPDATE',
            jsonb_build_object(
                'titulo', jsonb_build_object('old', OLD.titulo, 'new',
NEW.titulo),
                'precio', jsonb_build_object('old', OLD.precio_base, 'new',
NEW.precio_base),
                'categoria', jsonb_build_object('old', OLD.categoria, 'new',
NEW.categoria),
                'atributos', jsonb_build_object('old', OLD.atributos, 'new',
NEW.atributos)
            ),
            CURRENT_USER;
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO auditoria_eventos (evento_id, operacion, cambios, usuario)
        VALUES (OLD.id, 'DELETE', to_jsonb(OLD), CURRENT_USER);
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Trigger de auditoría
CREATE TRIGGER trg_audit_eventos
AFTER UPDATE OR DELETE ON eventos
FOR EACH ROW EXECUTE FUNCTION audit_eventos();
```

Paso 4: Insertar Datos de Prueba

sql

```
-- Insertar eventos con diferentes atributos
INSERT INTO eventos (id, titulo, descripcion, categoria, precio_base, atributos)
VALUES
    (uuidv7(), 'Concierto de Rock', 'Banda internacional en vivo', 'musica',
45.00,
     '{"artista": "The Rocks", "genero": "rock", "duracion": 120,
"clasificacion": "ATP"}'::jsonb),
    (uuidv7(), 'Conferencia Tech 2026', 'Evento de tecnología e innovación',
'conferencia', 299.00,
```

```
     '{"ponentes": ["Alice", "Bob"], "track": "backend", "nivel":
"intermedio"}'::jsonb),
    (uuidv7(), 'Taller de PostgreSQL', 'Aprende PostgreSQL 18', 'educacion',
0.00,
     '{"instructor": "Maria", "duracion_horas": 8, "material":
"included"}'::jsonb);

-- Insertar reservas
INSERT INTO reservas_eventos (evento_id, sala_id, horario, capacidad_maxima)
SELECT
    id,
    101,
    tstzrange('2026-05-15 19:00', '2026-05-15 22:00'),
    500
FROM eventos WHERE titulo = 'Concierto de Rock';
```

Paso 5: Consultas de Análisis

sql

```
-- 1. Buscar eventos por título (case-insensitive con índice)
SELECT * FROM eventos
WHERE casefold(titulo) = casefold('concierto de rock');

-- 2. Eventos con ciertos atributos JSON
SELECT titulo, categoria, atributos->>'artista' AS artista
FROM eventos
WHERE atributos @> '{"genero": "rock"}';

-- 3. Verificar solapamiento de reservas
SELECT e.titulo, r.sala_id, r.horario
FROM reservas_eventos r
JOIN eventos e ON r.evento_id = e.id
WHERE r.sala_id = 101
  AND r.horario && tstzrange('2026-05-15 20:00', '2026-05-15 21:00');

-- 4. Auditoría de cambios
SELECT * FROM auditoria_eventos ORDER BY fecha DESC LIMIT 10;

-- 5. Estadísticas de eventos por categoría usando JSONB
SELECT
    categoria,
    COUNT(*) as total,
    ROUND(AVG(precio_base), 2) as precio_promedio,
    jsonb_agg(atributos) as todos_atributos
FROM eventos
GROUP BY categoria;
```

Resumen del Módulo 5

Característica	Versiones anteriores	PostgreSQL 18
RETURNING	Solo NEW	OLD y NEW
Restricciones de solapamiento	EXCLUDE USING gist (sintaxis compleja)	WITHOUT OVERLAPS (sintaxis limpia)
UUID por defecto	UUIDv4 (aleatorio, fragmenta índices)	UUIDv7 (monotónico, amigable con caché)

Característica	Versiones anteriores	PostgreSQL 18
Columnas generadas	STORED por defecto	VIRTUAL por defecto
Collation para texto	libc o ICU (rendimiento variable)	PG_UNICODE_FAST (rápida + Unicode completo)
Comparación case-insensitive	ILIKE (sin índice fácil)	casefold() con índice
JSONB	Funcionalidad base	Operadores optimizados + GIN paralelo

---

Preparación para el Proyecto Final

Temas a integrar:

- Combinar monitoreo (Módulo 2) con optimización (Módulo 3)
- Aplicar seguridad OAuth (Módulo 4) a la aplicación de eventos
- Usar las nuevas capacidades JSONB para logging estructurado

**Lect

desarrolla el proyecto final

Proyecto Final: Sistema de Gestión de Eventos con PostgreSQL 18

Objetivo del Proyecto Final

Integrar todos los conocimientos adquiridos durante el curso para diseñar, implementar y optimizar un sistema completo de gestión de eventos utilizando las características avanzadas de PostgreSQL 18.

Descripción General

Desarrollarás una plataforma de gestión de eventos para una empresa de organización de conferencias que requiere:

- Gestión de eventos, ponentes, asistentes y reservas
- Sistema de auditoría completa con OLD/NEW
- Restricciones temporales para evitar solapamientos
- Autenticación OAuth 2.0 para usuarios externos

- Monitoreo y optimización de rendimiento
- Alta disponibilidad básica

Entorno: PostgreSQL 18 en contenedor Docker con extensiones habilitadas.

Duración estimada: 8-10 horas distribuidas en 5 entregas.

Estructura del Proyecto

Entrega 1: Diseño de Esquema y Arquitectura (2 horas)

Entrega 2: Implementación de Seguridad y Autenticación (2 horas)

Entrega 3: Desarrollo de Funcionalidad Core (2 horas)

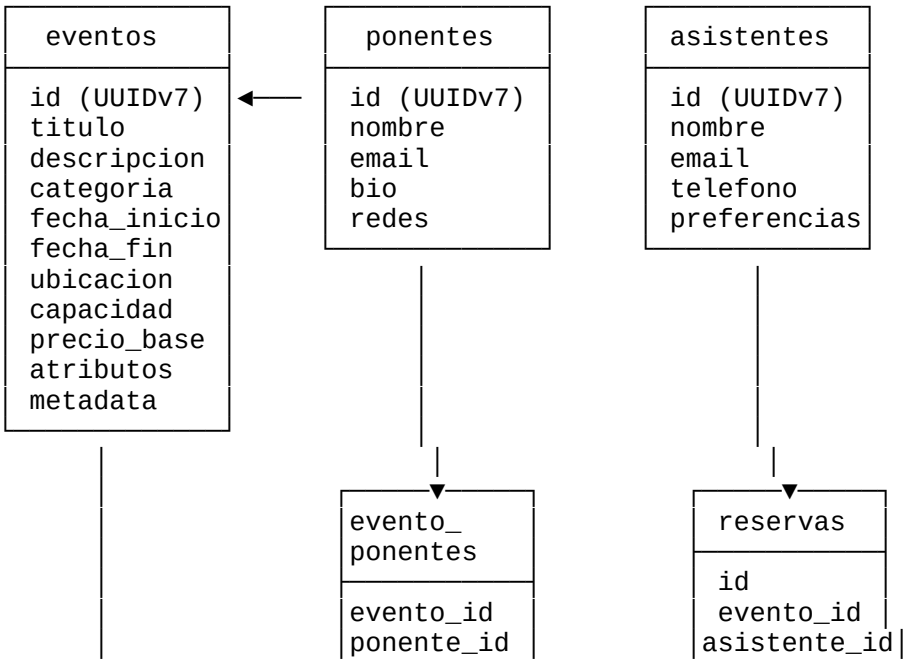
Entrega 4: Monitoreo y Optimización (2 horas)

Entrega 5: Documentación y Presentación (2 horas)

Entrega 1: Diseño de Esquema y Arquitectura (2 horas)

1.1. Diagrama Entidad-Relación

text



```
│                │rol          │          │ fecha     │
│                │horario      │          │ estado    │
└────────────────┴─────────────┘          │ pago_id   │
                                          └───────────┘
```

1.2. Creación del Esquema

sql

```
-- Crear esquema para el proyecto
CREATE SCHEMA IF NOT EXISTS event_platform;
SET search_path TO event_platform, public;

-- Habilitar extensiones necesarias
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_stat_statements";
CREATE EXTENSION IF NOT EXISTS "btree_gist";  -- Para WITHOUT OVERLAPS
```

Tabla: eventos (Core)

sql

```
CREATE TABLE eventos (
    -- PK con UUIDv7 para mejor rendimiento de índices
    id UUID PRIMARY KEY DEFAULT uuidv7(),

    -- Datos básicos con collation rápida
    titulo TEXT NOT NULL COLLATE "PG_UNICODE_FAST",
    descripcion TEXT COLLATE "PG_UNICODE_FAST",
    categoria TEXT NOT NULL,

    -- Fechas con rango temporal
    fecha_inicio TIMESTAMPTZ NOT NULL,
    fecha_fin TIMESTAMPTZ NOT NULL,
    CHECK (fecha_fin > fecha_inicio),

    -- Capacidad y precio
    ubicacion TEXT NOT NULL,
    capacidad_maxima INTEGER NOT NULL CHECK (capacidad_maxima > 0),
    precio_base DECIMAL(10,2) NOT NULL CHECK (precio_base >= 0),

    -- Datos semiestructurados
    atributos JSONB DEFAULT '{}'::jsonb,

    -- Metadatos con auditoría automática
    metadata JSONB DEFAULT jsonb_build_object(
        'creado_en', NOW(),
        'creado_por', CURRENT_USER,
        'modificado_en', NULL,
        'modificado_por', NULL,
        'versiones', 1
    ),

    -- Control de estado
    estado TEXT DEFAULT 'ACTIVO' CHECK (estado IN ('ACTIVO', 'CANCELADO',
'FINALIZADO')),

    -- Timestamps automáticos
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
```

```
    -- Restricción: no hay dos eventos en misma ubicación con fechas solapadas
    UNIQUE (ubicacion, tstzrange(fecha_inicio, fecha_fin) WITHOUT OVERLAPS)
);

-- Índices optimizados
CREATE INDEX idx_eventos_titulo_casefold ON eventos (casefold(titulo));
CREATE INDEX idx_eventos_categoria ON eventos (categoria);
CREATE INDEX idx_eventos_fechas ON eventos (fecha_inicio, fecha_fin);
CREATE INDEX idx_eventos_atributos_gin ON eventos USING gin (atributos);
CREATE INDEX idx_eventos_estado ON eventos (estado);
```

Tabla: ponentes

sql

```
CREATE TABLE ponentes (
    id UUID PRIMARY KEY DEFAULT uuidv7(),
    nombre TEXT NOT NULL COLLATE "PG_UNICODE_FAST",
    email TEXT NOT NULL UNIQUE,
    bio TEXT,
    especialidad TEXT[] DEFAULT '{}',
    redes_sociales JSONB DEFAULT '{}'::jsonb,
    metadata JSONB DEFAULT jsonb_build_object(
        'creado_en', NOW(),
        'creado_por', CURRENT_USER
    ),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_ponentes_email ON ponentes (email);
CREATE INDEX idx_ponentes_nombre_casefold ON ponentes (casefold(nombre));
CREATE INDEX idx_ponentes_especialidad ON ponentes USING gin (especialidad);
```

Tabla: asistentes

sql

```
CREATE TABLE asistentes (
    id UUID PRIMARY KEY DEFAULT uuidv7(),
    nombre TEXT NOT NULL COLLATE "PG_UNICODE_FAST",
    email TEXT NOT NULL UNIQUE,
    telefono TEXT,
    preferencias JSONB DEFAULT '{"notificaciones": true, "idioma":
"es"}'::jsonb,

    -- OAuth externo
    oauth_provider TEXT,
    oauth_user_id TEXT,
    UNIQUE (oauth_provider, oauth_user_id),

    metadata JSONB DEFAULT jsonb_build_object(
        'registro_en', NOW(),
        'ultimo_acceso', NULL
    ),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_asistentes_email ON asistentes (email);
CREATE INDEX idx_asistentes_preferencias_gin ON asistentes USING gin
(preferencias);
```

Tabla: evento_ponentes (Relación muchos a muchos con rol)

sql

```
CREATE TABLE evento_ponentes (
    evento_id UUID NOT NULL REFERENCES eventos(id) ON DELETE CASCADE,
    ponente_id UUID NOT NULL REFERENCES ponentes(id) ON DELETE CASCADE,
    rol TEXT DEFAULT 'PONENTE' CHECK (rol IN ('PONENTE', 'MODERADOR',
'PANELISTA')),
    horario_asignado TSTZRANGE,
    notas TEXT,

    PRIMARY KEY (evento_id, ponente_id),

    -- Un ponente no puede estar en dos roles en el mismo evento
    UNIQUE (evento_id, ponente_id, rol)
);
```

Tabla: reservas (con pago simulado)

sql

```
CREATE TABLE reservas (
    id UUID PRIMARY KEY DEFAULT uuidv7(),
    evento_id UUID NOT NULL REFERENCES eventos(id) ON DELETE CASCADE,
    asistente_id UUID NOT NULL REFERENCES asistentes(id) ON DELETE CASCADE,

    fecha_reserva TIMESTAMPTZ DEFAULT NOW(),
    cantidad INTEGER NOT NULL DEFAULT 1 CHECK (cantidad > 0),
    precio_unitario DECIMAL(10,2) NOT NULL,
    precio_total DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario)
STORED,

    estado TEXT DEFAULT 'PENDIENTE' CHECK (estado IN ('PENDIENTE', 'CONFIRMADA',
'CANCELADA', 'COMPLETADA')),

    -- Datos de pago (simulado)
    pago_id TEXT,
    pago_metadata JSONB DEFAULT '{}'::jsonb,

    metadata JSONB DEFAULT jsonb_build_object('creado_en', NOW()),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),

    -- Restricción: un asistente no puede reservar dos eventos solapados
    UNIQUE (asistente_id, tstzrange(
        (SELECT fecha_inicio FROM eventos WHERE id = evento_id),
        (SELECT fecha_fin FROM eventos WHERE id = evento_id)
    ) WITHOUT OVERLAPS)
);

CREATE INDEX idx_reservas_evento ON reservas (evento_id);
CREATE INDEX idx_reservas_asistente ON reservas (asistente_id);
CREATE INDEX idx_reservas_estado ON reservas (estado);
```

Tabla de Auditoría

sql

```
CREATE TABLE auditoria_eventos (
    id SERIAL PRIMARY KEY,
    tabla_afectada TEXT NOT NULL,
```

```
    registro_id UUID NOT NULL,
    operacion TEXT NOT NULL CHECK (operacion IN ('INSERT', 'UPDATE', 'DELETE')),
    valores_anteriores JSONB,
    valores_nuevos JSONB,
    usuario TEXT NOT NULL,
    ip_address INET,
    user_agent TEXT,
    fecha TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_auditoria_registro ON auditoria_eventos (tabla_afectada,
registro_id);
CREATE INDEX idx_auditoria_fecha ON auditoria_eventos (fecha DESC);
```

1.3. Función de Actualización Automática de updated_at

sql

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Aplicar trigger a todas las tablas
CREATE TRIGGER update_eventos_updated_at BEFORE UPDATE ON eventos
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_ponentes_updated_at BEFORE UPDATE ON ponentes
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_asistentes_updated_at BEFORE UPDATE ON asistentes
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_reservas_updated_at BEFORE UPDATE ON reservas
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

1.4. Trigger de Auditoría con OLD/NEW

sql

```
CREATE OR REPLACE FUNCTION audit_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO auditoria_eventos (
        tabla_afectada, registro_id, operacion,
        valores_anteriores, valores_nuevos,
        usuario, ip_address, user_agent
    )
    VALUES (
        TG_TABLE_NAME,
        COALESCE(NEW.id, OLD.id),
        TG_OP,
        CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN to_jsonb(OLD) ELSE NULL
END,
        CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN to_jsonb(NEW) ELSE NULL
END,
        CURRENT_USER,
        inet_client_addr(),
```

```
        current_setting('application_name', true)
    );

    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

-- Aplicar a tablas críticas
CREATE TRIGGER audit_eventos AFTER INSERT OR UPDATE OR DELETE ON eventos
    FOR EACH ROW EXECUTE FUNCTION audit_trigger_function();

CREATE TRIGGER audit_reservas AFTER INSERT OR UPDATE OR DELETE ON reservas
    FOR EACH ROW EXECUTE FUNCTION audit_trigger_function();
```

Entregable 1:

- Script SQL completo de creación del esquema
- Diagrama ER en formato PDF o PNG
- Explicación de decisiones de diseño (UUIDv7, WITHOUT OVERLAPS, JSONB)

Entrega 2: Implementación de Seguridad y Autenticación (2 horas)

2.1. Configuración de TLS 1.3

bash

```
# Generar certificados para desarrollo
mkdir -p /var/lib/postgresql/certs
cd /var/lib/postgresql/certs

# Generar CA
openssl req -new -x509 -days 365 -nodes -text \
    -out ca.crt -keyout ca.key \
    -subj "/CN=event-platform-ca"

# Generar certificado del servidor
openssl req -new -nodes -text -out server.csr \
    -keyout server.key -subj "/CN=postgres.event-platform.local"
openssl x509 -req -in server.csr -days 365 -CA ca.crt -CAkey ca.key \
    -CAcreateserial -out server.crt

chmod 600 server.key
chown postgres:postgres server.key
```

conf

```
# postgresql.conf
ssl = on
ssl_cert_file = '/var/lib/postgresql/certs/server.crt'
ssl_key_file = '/var/lib/postgresql/certs/server.key'
ssl_ca_file = '/var/lib/postgresql/certs/ca.crt'
ssl_min_protocol_version = 'TLSv1.2'
ssl_max_protocol_version = 'TLSv1.3'
```

```
ssl_tls13_ciphers = 'TLS_AES_256_GCM_SHA384'
ssl_ciphers = 'HIGH:!aNULL:!MD5:!3DES'
```

2.2. Roles y Permisos

sql

```
-- Crear roles de aplicación
CREATE ROLE app_admin WITH LOGIN PASSWORD 'admin_secure_2026!';
CREATE ROLE app_organizador WITH LOGIN PASSWORD 'org_secure_2026!';
CREATE ROLE app_asistente WITH LOGIN PASSWORD 'asist_secure_2026!';
CREATE ROLE app_readonly WITH LOGIN PASSWORD 'read_secure_2026!';

-- Crear roles de grupo
CREATE ROLE grupo_administradores;
CREATE ROLE grupo_organizadores;
CREATE ROLE grupo_asistentes;
CREATE ROLE grupo_lectores;

-- Asignar miembros
GRANT grupo_administradores TO app_admin;
GRANT grupo_organizadores TO app_organizador;
GRANT grupo_asistentes TO app_asistente;
GRANT grupo_lectores TO app_readonly;

-- Permisos por esquema
GRANT USAGE ON SCHEMA event_platform TO grupo_organizadores, grupo_asistentes,
grupo_lectores;
GRANT ALL PRIVILEGES ON SCHEMA event_platform TO grupo_administradores;

-- Permisos específicos por tabla
-- Administradores: todo
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA event_platform TO
grupo_administradores;

-- Organizadores: CRUD en eventos y ponentes
GRANT SELECT, INSERT, UPDATE, DELETE ON eventos, ponentes, evento_ponentes TO
grupo_organizadores;
GRANT SELECT ON reservas, asistentes TO grupo_organizadores;

-- Asistentes: solo sus propias reservas
GRANT SELECT ON eventos TO grupo_asistentes;
GRANT SELECT, INSERT, UPDATE ON reservas TO grupo_asistentes;
GRANT SELECT ON asistentes TO grupo_asistentes;

-- Lectores: solo SELECT
GRANT SELECT ON ALL TABLES IN SCHEMA event_platform TO grupo_lectores;

-- Secuencias
GRANT USAGE ON ALL SEQUENCES IN SCHEMA event_platform TO grupo_administradores,
grupo_organizadores;

-- Políticas RLS para asistentes (solo ven sus propios datos)
ALTER TABLE asistentes ENABLE ROW LEVEL SECURITY;
ALTER TABLE reservas ENABLE ROW LEVEL SECURITY;

CREATE POLICY asistentes_self_policy ON asistentes
    USING (id = current_setting('app.current_asistente_id', true)::UUID);

CREATE POLICY reservas_self_policy ON reservas
```

```
    USING (asistente_id = current_setting('app.current_asistente_id',
true)::UUID);
```

2.3. Migración de MD5 a SCRAM-SHA-256

sql

```
-- Verificar usuarios con MD5
SELECT usename FROM pg_shadow WHERE passwd LIKE 'md5%';

-- Migrar contraseñas (forzar cambio en próximo login)
ALTER USER app_admin PASSWORD 'admin_secure_2026!';
ALTER USER app_organizador PASSWORD 'org_secure_2026!';
ALTER USER app_asistente PASSWORD 'asist_secure_2026!';
ALTER USER app_readonly PASSWORD 'read_secure_2026!';

-- Configurar SCRAM como default
ALTER SYSTEM SET password_encryption = 'scram-sha-256';
SELECT pg_reload_conf();
```

2.4. Función de Autenticación con JWT (Simulado)

sql

```
-- Función para validar token JWT (simplificada para demo)
CREATE OR REPLACE FUNCTION authenticate_with_jwt(token TEXT)
RETURNS TABLE(asistente_id UUID, email TEXT, nombre TEXT) AS $$
DECLARE
    payload JSONB;
BEGIN
    -- Simular decodificación JWT (en producción usar extensión pgjwt)
    payload := decode(token, 'base64')::jsonb;

    -- Buscar asistente por email
    RETURN QUERY
    SELECT id, email, nombre
    FROM asistentes
    WHERE email = payload->>'email';

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Usuario no encontrado';
    END IF;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Entregable 2:

- Archivos de configuración TLS
- Script de creación de roles y permisos
- Verificación de conexión segura con `psql "sslmode=require"`

Entrega 3: Desarrollo de Funcionalidad Core (2 horas)

3.1. CRUD de Eventos con Validaciones

sql

```
-- Función para crear evento con validaciones
CREATE OR REPLACE FUNCTION crear_evento(
    p_titulo TEXT,
    p_descripcion TEXT,
    p_categoria TEXT,
    p_fecha_inicio TIMESTAMPTZ,
    p_fecha_fin TIMESTAMPTZ,
    p_ubicacion TEXT,
    p_capacidad_maxima INTEGER,
    p_precio_base DECIMAL(10,2),
    p_atributos JSONB DEFAULT '{}'
)
RETURNS UUID AS $$
DECLARE
    v_evento_id UUID;
BEGIN
    -- Validaciones
    IF p_fecha_inicio <= NOW() THEN
        RAISE EXCEPTION 'La fecha de inicio debe ser futura';
    END IF;

    IF p_capacidad_maxima <= 0 THEN
        RAISE EXCEPTION 'La capacidad debe ser mayor a 0';
    END IF;

    IF p_precio_base < 0 THEN
        RAISE EXCEPTION 'El precio no puede ser negativo';
    END IF;

    -- Verificar solapamiento de ubicación (automático por constraint)
    INSERT INTO eventos (
        id, titulo, descripcion, categoria,
        fecha_inicio, fecha_fin, ubicacion,
        capacidad_maxima, precio_base, atributos
    ) VALUES (
        uuidv7(), p_titulo, p_descripcion, p_categoria,
        p_fecha_inicio, p_fecha_fin, p_ubicacion,
        p_capacidad_maxima, p_precio_base, p_atributos
    )
    RETURNING id INTO v_evento_id;

    RETURN v_evento_id;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Ejemplo de uso
SELECT crear_evento(
    'PostgreSQL 18 Summit 2026',
    'La conferencia anual de PostgreSQL',
    'conferencia',
    '2026-10-15 09:00:00+00',
```

```
    '2026-10-17 18:00:00+00',
    'Centro de Convenciones, Madrid',
    500,
    299.99,
    '{"ponentes_confimados": [], "patrocinadores": []}'::jsonb
);
```

3.2. Sistema de Reservas con RETURNING OLD/NEW

sql

```sql
-- Función para crear reserva con auditoría explícita
CREATE OR REPLACE FUNCTION crear_reserva(
    p_evento_id UUID,
    p_asistente_id UUID,
    p_cantidad INTEGER
)
RETURNS TABLE(
    reserva_id UUID,
    evento_titulo TEXT,
    precio_total DECIMAL(10,2),
    estado TEXT,
    asistentes_restantes BIGINT
) AS $$
DECLARE
    v_evento RECORD;
    v_reserva_id UUID;
    v_reservas_actuales BIGINT;
BEGIN
    -- Obtener información del evento con lock para evitar overbooking
    SELECT * INTO v_evento
    FROM eventos
    WHERE id = p_evento_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Evento no encontrado';
    END IF;

    IF v_evento.estado != 'ACTIVO' THEN
        RAISE EXCEPTION 'Evento no está activo';
    END IF;

    -- Contar reservas confirmadas actuales
    SELECT COALESCE(SUM(cantidad), 0) INTO v_reservas_actuales
    FROM reservas
    WHERE evento_id = p_evento_id
      AND estado IN ('PENDIENTE', 'CONFIRMADA');

    IF v_reservas_actuales + p_cantidad > v_evento.capacidad_maxima THEN
        RAISE EXCEPTION 'Capacidad excedida. Disponibles: %',
            v_evento.capacidad_maxima - v_reservas_actuales;
    END IF;

    -- Crear reserva
    INSERT INTO reservas (
        id, evento_id, asistente_id, cantidad,
        precio_unitario, estado
    ) VALUES (
        uuidv7(), p_evento_id, p_asistente_id, p_cantidad,
        v_evento.precio_base, 'CONFIRMADA'
```

```
    )
    RETURNING id INTO v_reserva_id;

    -- Retornar información usando RETURNING implícito
    RETURN QUERY
    SELECT
        r.id,
        e.titulo,
        r.precio_total,
        r.estado,
        e.capacidad_maxima - COALESCE(
            (SELECT SUM(cantidad) FROM reservas
             WHERE evento_id = p_evento_id AND estado IN ('PENDIENTE',
'CONFIRMADA')), 0
        ) AS asistentes_restantes
    FROM reservas r
    JOIN eventos e ON r.evento_id = e.id
    WHERE r.id = v_reserva_id;

END;
$$ LANGUAGE plpgsql;
```

3.3. Consultas de Reportes con JSONB

sql

```
-- Reporte de popularidad de eventos por categoría
CREATE OR REPLACE VIEW reporte_popularidad_eventos AS
SELECT
    e.categoria,
    COUNT(DISTINCT e.id) as total_eventos,
    SUM(r.cantidad) as total_reservas,
    ROUND(AVG(r.precio_unitario), 2) as precio_promedio,
    jsonb_agg(
        jsonb_build_object(
            'evento', e.titulo,
            'reservas', r.cantidad,
            'fecha', e.fecha_inicio
        )
    ) FILTER (WHERE r.cantidad > 10) as eventos_populares
FROM eventos e
LEFT JOIN reservas r ON e.id = r.evento_id AND r.estado = 'CONFIRMADA'
WHERE e.estado = 'ACTIVO'
GROUP BY e.categoria;

-- Reporte de asistentes frecuentes
CREATE OR REPLACE VIEW reporte_asistentes_frecuentes AS
SELECT
    a.id,
    a.nombre,
    a.email,
    COUNT(r.id) as eventos_asistidos,
    SUM(r.cantidad) as entradas_compradas,
    SUM(r.precio_total) as gasto_total,
    a.preferencias
FROM asistentes a
JOIN reservas r ON a.id = r.asistente_id
WHERE r.estado = 'CONFIRMADA'
GROUP BY a.id, a.nombre, a.email, a.preferencias
HAVING COUNT(r.id) >= 2
ORDER BY eventos_asistidos DESC;
```

3.4. Función de Cancelación con OLD/NEW

sql

```
-- Cancelar reserva con registro de cambios
CREATE OR REPLACE FUNCTION cancelar_reserva(p_reserva_id UUID)
RETURNS JSONB AS $$
DECLARE
    v_reserva RECORD;
    v_resultado JSONB;
BEGIN
    -- Obtener reserva antes de cancelar
    SELECT * INTO v_reserva
    FROM reservas
    WHERE id = p_reserva_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Reserva no encontrada';
    END IF;

    IF v_reserva.estado = 'CANCELADA' THEN
        RAISE EXCEPTION 'Reserva ya está cancelada';
    END IF;

    -- Cancelar con RETURNING OLD/NEW explícito
    UPDATE reservas
    SET estado = 'CANCELADA',
        metadata = metadata || jsonb_build_object(
            'cancelada_en', NOW(),
            'cancelada_por', CURRENT_USER,
            'precio_original', v_reserva.precio_total
        )
    WHERE id = p_reserva_id
    RETURNING jsonb_build_object(
        'reserva_id', id,
        'estado_anterior', OLD.estado,
        'estado_nuevo', NEW.estado,
        'fecha_cancelacion', NOW(),
        'usuario', CURRENT_USER
    ) INTO v_resultado;

    RETURN v_resultado;
END;
$$ LANGUAGE plpgsql;
```

Entregable 3:

- Scripts de funciones CRUD
- Pruebas unitarias de las funciones
- Capturas de ejecución de consultas

Entrega 4: Monitoreo y Optimización (2 horas)

4.1. Configuración de pg_stat_statements

sql

```
-- Habilitar pg_stat_statements
ALTER SYSTEM SET shared_preload_libraries = 'pg_stat_statements';
ALTER SYSTEM SET pg_stat_statements.track = 'all';
ALTER SYSTEM SET pg_stat_statements.max = 10000;
ALTER SYSTEM SET pg_stat_statements.track_utility = on;

-- Reiniciar PostgreSQL (fuera de SQL)
-- SELECT pg_reload_conf(); no es suficiente, requiere reinicio
```

4.2. Vista de Monitoreo Personalizada

sql

```
CREATE OR REPLACE VIEW dashboard_rendimiento AS
SELECT
    'Consultas lentas (> 1 segundo)' AS metric,
    COUNT(*) AS value,
    ROUND(AVG(mean_exec_time), 2) AS avg_time_ms
FROM pg_stat_statements
WHERE mean_exec_time > 1000
UNION ALL
SELECT
    'Índices no utilizados',
    COUNT(*)::text,
    SUM(pg_relation_size(indexrelid))::numeric
FROM pg_stat_user_indexes
WHERE idx_scan = 0
UNION ALL
SELECT
    'Tablas con dead tuples > 5%',
    COUNT(*)::text,
    ROUND(AVG(n_dead_tup * 100.0 / NULLIF(n_live_tup, 0)), 2)
FROM pg_stat_user_tables
WHERE n_dead_tup > 0
  AND (n_dead_tup * 100.0 / NULLIF(n_live_tup, 0)) > 5
UNION ALL
SELECT
    'Eventos próximos a wraparound',
    COUNT(*)::text,
    MAX(age(relfrozenxid))::numeric
FROM pg_class
WHERE relfrozenxid IS NOT NULL
  AND age(relfrozenxid) > 1000000000;
```

4.3. Optimización de Consultas Problemáticas

sql

```
-- Identificar consultas que generan más I/O
SELECT
    queryid,
    LEFT(query, 100) AS query_preview,
    calls,
```

```
    total_exec_time,
    mean_exec_time,
    shared_blks_read,
    shared_blks_hit,
    ROUND(100.0 * shared_blks_hit / NULLIF(shared_blks_hit + shared_blks_read,
0), 2) AS cache_hit_ratio
FROM pg_stat_statements
WHERE shared_blks_read > 1000
ORDER BY shared_blks_read DESC
LIMIT 10;

-- Crear índice faltante detectado
-- Ejemplo: si muchas consultas filtran por ubicacion y fecha
CREATE INDEX CONCURRENTLY idx_eventos_ubicacion_fechas
ON eventos (ubicacion, fecha_inicio, fecha_fin);
```

4.4. Verificación de WITHOUT OVERLAPS

sql

```
-- Probar restricción de solapamiento
-- Esto debe fallar:
INSERT INTO eventos (id, titulo, fecha_inicio, fecha_fin, ubicacion, categoria,
capacidad_maxima, precio_base)
VALUES (
    uuidv7(),
    'Evento Conflictivo',
    '2026-10-15 10:00:00+00',
    '2026-10-15 12:00:00+00',
    'Centro de Convenciones, Madrid',  -- Misma ubicación que evento existente
    'test',
    100,
    0
);

-- Verificar constraint usando pg_constraint
SELECT
    conname,
    contype,
    pg_get_constraintdef(oid) AS constraint_def
FROM pg_constraint
WHERE conname LIKE '%without_overlaps%';
```

4.5. Script de Monitoreo Continuo

bash

```
#!/bin/bash
# monitor_postgres.sh

while true; do
    echo "=== $(date) ===" >> /var/log/postgres_monitor.log

    psql -d event_platform -U app_readonly -t -c "
    SELECT
        NOW() as timestamp,
        (SELECT count(*) FROM pg_stat_activity WHERE state = 'active') as
active_connections,
        (SELECT ROUND(SUM(shared_blks_read)::numeric / 1024, 2) FROM
pg_stat_statements) as total_mb_read,
```

```
        (SELECT COUNT(*) FROM reservas WHERE estado = 'PENDIENTE' AND created_at
< NOW() - interval '10 minutes') as stale_reservations
    " >> /var/log/postgres_monitor.log

    sleep 60
done
```

Entregable 4:

- Configuración de monitoreo funcionando
- Capturas de dashboard de rendimiento
- Plan de optimización aplicado

Entrega 5: Documentación y Presentación (2 horas)

5.1. Documentación del Sistema

Archivo README.md

markdown

```
# Sistema de Gestión de Eventos - PostgreSQL 18

## Arquitectura
- PostgreSQL 18 con UUIDv7, WITHOUT OVERLAPS, JSONB
- Autenticación SCRAM-SHA-256 + TLS 1.3
- Auditoría completa con OLD/NEW

## Instalación Rápida
```bash
docker run -d --name pg18-eventos \
  -e POSTGRES_PASSWORD=secure \
  -e POSTGRES_DB=event_platform \
  -v $(pwd)/scripts:/docker-entrypoint-initdb.d \
  postgres:18
```

API de Funciones

Función	Descripción	Parámetros
crear_evento()	Crea nuevo evento	titulo, fecha_inicio, ubicacion, ...
crear_reserva()	Reserva entradas	evento_id, asistente_id, cantidad
cancelar_reserva()	Cancela reserva	reserva_id

Monitoreo

- Vista: dashboard_rendimiento
```

- Logs: /var/log/postgres_monitor.log
- Métricas clave: cache hit ratio, dead tuples, active connections

Seguridad

- TLS 1.3 con AES-256-GCM
- SCRAM-SHA-256 para contraseñas
- RLS para asistentes

text

```
## 5.2. Pruebas de Carga

```sql
-- Generar datos de prueba masivos
DO $$
BEGIN
    FOR i IN 1..10000 LOOP
        INSERT INTO asistentes (id, nombre, email, preferencias)
        VALUES (
            uuidv7(),
            'Asistente ' || i,
            'asistente' || i || '@test.com',
            '{"notificaciones": true, "intereses": ["tech", "data"]}'::jsonb
        );
    END LOOP;

    FOR i IN 1..1000 LOOP
        PERFORM crear_evento(
            'Evento de prueba ' || i,
            'Descripción automática',
            'test',
            NOW() + (i || ' days')::interval,
            NOW() + (i || ' days')::interval + interval '2 hours',
            'Sala ' || (i % 100),
            100 + (i % 900),
            49.99,
            '{"tipo": "automático"}'::jsonb
        );
    END LOOP;
END $$;
```

5.3. Script de Validación Final

sql

```
-- Verificar todas las funcionalidades implementadas
DO $$
DECLARE
    v_evento_id UUID;
    v_asistente_id UUID;
    v_reserva_id UUID;
BEGIN
    -- 1. Crear evento
    SELECT crear_evento(
        'Evento Validación Final',
```
```

```
        'Validando todas las funcionalidades',
        'validación',
        NOW() + interval '7 days',
        NOW() + interval '7 days' + interval '3 hours',
        'Sala Validación',
        100,
        99.99,
        '{"validacion": true}'::jsonb
    ) INTO v_evento_id;
    RAISE NOTICE 'Evento creado: %', v_evento_id;

    -- 2. Crear asistente
    INSERT INTO asistentes (id, nombre, email)
    VALUES (uuidv7(), 'Usuario Test', 'test@validacion.com')
    RETURNING id INTO v_asistente_id;
    RAISE NOTICE 'Asistente creado: %', v_asistente_id;

    -- 3. Crear reserva
    SELECT * INTO v_reserva_id FROM crear_reserva(v_evento_id, v_asistente_id,
2);
    RAISE NOTICE 'Reserva creada: %', v_reserva_id;

    -- 4. Cancelar reserva
    PERFORM cancelar_reserva(v_reserva_id);
    RAISE NOTICE 'Reserva cancelada';

    -- 5. Verificar auditoría
    IF EXISTS (SELECT 1 FROM auditoria_eventos WHERE registro_id = v_evento_id)
THEN
        RAISE NOTICE 'Auditoría funcionando correctamente';
    END IF;

    RAISE NOTICE 'VALIDACIÓN COMPLETADA EXITOSAMENTE';
END $$;
```

5.4. Presentación Final

Preparar una presentación de 10 minutos incluyendo:

1. **Arquitectura del sistema** (2 min)

- Diagrama ER
- Decisiones técnicas (UUIDv7, WITHOUT OVERLAPS)

2. **Características innovadoras de PG 18** (3 min)

- OLD/NEW en RETURNING
- Restricciones temporales nativas
- PG_UNICODE_FAST

3. **Monitoreo y rendimiento** (2 min)

- Dashboard creado
- Optimizaciones aplicadas
- Métricas clave

4. **Seguridad implementada** (2 min)

- TLS 1.3
- SCRAM-SHA-256
- RLS

5. **Demostración en vivo** (1 min)

- Ejecutar script de validación
- Mostrar dashboard

Entregable 5:

- Documentación completa en formato Markdown
- Script de validación funcional
- Presentación en PDF
- Enlace a repositorio GitHub (opcional)

Criterios de Evaluación

Criterio	Peso	Descripción
Completitud del esquema	20%	Todas las tablas, relaciones, constraints implementadas
Uso de características PG 18	25%	UUIDv7, WITHOUT OVERLAPS, OLD/NEW, PG_UNICODE_FAST
Seguridad	15%	TLS, SCRAM, RLS, auditoría funcionando
Monitoreo	15%	Dashboard, pg_stat_statements, scripts
Optimización	15%	Índices correctos, consultas eficientes, análisis EXPLAIN
Documentación	10%	Clara, completa, reproducible

Entregables Finales

text

```
proyecto_final/
├── scripts/
│   ├── 01_schema.sql
│   ├── 02_security.sql
│   ├── 03_functions.sql
│   ├── 04_monitoring.sql
│   └── 05_validation.sql
├── config/
│   ├── postgresql.conf
```

```
│   ├── pg_hba.conf
│   └── certs/
├── docs/
│   ├── README.md
│   ├── DIAGRAMA_ER.pdf
│   └── PRESENTACION.pdf
├── logs/
│   └── monitor.log
└── docker-compose.yml
```

Comandos para Ejecutar el Proyecto Completo

bash

```
# 1. Clonar/crear estructura
mkdir proyecto_final && cd proyecto_final

# 2. Iniciar contenedor
docker-compose up -d

# 3. Ejecutar scripts en orden
for script in scripts/0*.sql; do
    psql -h localhost -U postgres -d event_platform -f $script
done

# 4. Ejecutar validación
psql -h localhost -U postgres -d event_platform -f scripts/05_validation.sql

# 5. Verificar monitoreo
psql -h localhost -U app_readonly -d event_platform -c "SELECT * FROM
dashboard_rendimiento;"
```

¡Felicidades! Has completado el curso de PostgreSQL 18

Al finalizar este proyecto, habrás demostrado competencia en:

- Arquitectura avanzada de PostgreSQL 18
- Implementación de características de vanguardia (UUIDv7, WITHOUT OVERLAPS)
- Monitoreo y optimización en entornos reales
- Seguridad de nivel empresarial
- Desarrollo de aplicaciones complejas con SQL avanzado

Próximos pasos sugeridos:

- Implementar replicación lógica con pglogical
- Explorar sharding con pg_pathman
- Contribuir a la comunidad PostgreSQL